Structural Assurability: A Claim-Relative Framework for Evidence-Based AI Assurance

Denise M. Case

School of Computer Science and Information Systems
Northwest Missouri State University, Maryville, MO, USA

Abstract

Structural assurance claims depend not only on what an AI system does, but on what evidence its architecture makes practically obtainable about what it does. Two systems can exhibit equivalent functional behavior while differing materially in their capacity to support or refute the same assurance claim. This paper formalizes that difference as *structural assurability*: the claim-relative capacity of a system and its evidence-producing environment to make assurance-relevant evidence obtainable under specified access, cooperation, and resource constraints. The framework separates structural properties from evidence quality and downstream assurance reasoning. It identifies seven provisional structural dimensions: observability, controllability, traceability, reconstructability, independence, integrity, and evidence coverage; and introduces a separating-construction criterion: a dimension is assurance-relevant when removing it can eliminate a distinction between systems that matters in at least one assurance context. Rather than reducing assurability to a scalar score, the framework compares systems through the claim-relevant distinctions their obtainable evidence enables, permitting equivalence, strict dominance, and incomparability. Representative AI architectures show that richer instrumentation can increase assurability for some claims, add nothing for others, and trade

internal visibility against independent evidence in ways that prevent a total ordering. The framework is deliberately open-ended: the proposed dimensions are neither claimed sufficient nor necessary in every context, and additional dimensions may be admitted when they support new separating constructions. Structural assurability therefore characterizes capacity for examination, not a favorable assurance outcome.

Keywords: AI assurance; structural assurability; assurance evidence; observability; traceability; auditability; runtime assurance; assurance cases

1 Introduction

Assurance depends on evidence.

For a claim that a system is safe, secure, reliable, compliant, or otherwise trustworthy, an evaluator must be able to obtain evidence bearing on that claim and connect the evidence to the claim through a defensible argument. Assurance-case methods make this relationship explicit: claims are supported by evidence through structured reasoning, while separate questions concern the adequacy of the argument and the confidence warranted by it Goodenough et al. (2012); Bloomfield and Bishop (2010).

For increasingly capable AI systems, however, an important question arises before the assurance argument can even be constructed:

> What evidence does the structure of the system make possible to obtain?

Two AI systems may exhibit equivalent externally measured performance while differing substantially in their capacity to expose evidence concerning how that performance was produced. One system may expose only prompts and outputs. Another may additionally preserve tool invocations, retrieval events, authorization decisions, state transitions, provenance, independently generated records, and selected internal-state measurements. Even when the systems behave identically over a specified evaluation suite, they need not support the same assurance inquiries.

This distinction becomes increasingly important as AI systems move from isolated prediction toward persistent, tool-using, stateful, and agentic operation. Pre-

deployment evaluation remains valuable, but deployed systems operate under changing inputs, environments, integrations, and incentives. Recent work has accordingly expanded from benchmark evaluation toward runtime monitoring, agent observability, auditability, provenance, assurance cases, interpretability, and internal-state monitoring National Institute of Standards and Technology (2026a); Fernsel et al. (2024); Guan et al. (2025); National Institute of Standards and Technology (2026b). These efforts expose a common underlying problem: assurance depends not merely on the existence of evaluation methods, but on whether the system and its operating environment make the evidence required by those methods obtainable.

This problem has mature analogues elsewhere in engineering.

Hardware design for testability recognizes that testing effectiveness is partly determined by the structure of the device being tested. Observability and controllability can be deliberately designed into hardware so that relevant internal states can be established and examined economically Abramovici et al. (1990); Williams and Parker (1982). Software architecture similarly affects testability, analyzability, diagnosability, and the practical cost of verification Bass et al. (2021); Clements et al. (2002); Kazman et al. (2000). Safety engineering recognizes that architecture affects the generation and independence of assurance evidence, while security engineering emphasizes provenance, tamper resistance, privilege separation, and evidence integrity under adversarial conditions SAE International (2010); RTCA (2011); Saltzer and Schroeder (1975); Anderson (2020).

AI assurance draws on all of these traditions, but the corresponding structural property is not usually the primary object of analysis.

A system may be described as auditable, monitorable, interpretable, observable, testable, or capable of supporting an assurance case. Each concept captures an important part of the problem. None alone answers the more general comparative question considered here:

> Given two AI systems subject to the same assurance claim, what structural differences determine whether one permits assurance-relevant evidence to be obtained that the other does not?

We call this property *structural assurability*.

Structural assurability is the claim-relative capacity of a system and its evidence-

producing environment to make evidence relevant to supporting or refuting an assurance claim practically obtainable under specified access, cooperation, and resource constraints.

We write this dependence schematically as

$$\mathcal{A}(S, C, K, B), \tag{1}$$

where S is the system, C is the assurance claim, K represents applicable knowledge, access, trust, and cooperation conditions, and B represents practical resource constraints.

The formulation is intentionally claim-relative.

A system that exposes extensive internal state may be highly assurable for a claim concerning internal computation while gaining little advantage for a claim resolved completely from an externally observable output. Likewise, an architecture with strong independent external monitoring may support some claims better than an architecture offering richer but operator-controlled internal telemetry.

Accordingly,

$$\text{assurability}(S) \tag{2}$$

without a specified assurance context is generally underspecified.

Structural assurability is also deliberately distinct from safety or other favorable system properties.

A system can be structurally highly assurable and nevertheless fail the assurance claim under investigation. Richer evidence may make failure easier to establish. Structural assurability concerns capacity for examination rather than the result of examination:

$$\boxed{\begin{aligned}&\text{Property satisfaction} \neq \text{Structural assurability} \\ &\qquad\qquad \neq \text{Assurance argument} \neq \text{Confidence in the argument.}\end{aligned}} \tag{3}$$

This separation is important because otherwise evidence-producing mechanisms can be conflated with evidence that happens to support a desired conclusion. A useful assurance structure should support both confirmation and refutation.

The central premise of this paper is therefore modest:

> Systems with equivalent functional behavior or measured performance can differ materially in assurability because their architectures and evidence-producing environments can differ in their capacity to make assurance-relevant distinctions possible.

The phrase *assurance-relevant distinctions* provides the core interpretation developed in this work.

Consider a claim concerning whether an AI agent performed an unauthorized consequential action. Relevant alternatives may include no action, authorized agent action, unauthorized agent action, human-initiated action, externally initiated action, or an action whose historical record has been altered. An evidence-producing structure contributes to assurability when it makes one or more such alternatives distinguishable where they would otherwise remain unresolved.

This formulation avoids equating assurability with evidence volume.

More logs do not necessarily imply greater assurability. More internal state does not necessarily imply greater assurability. More evaluation access does not necessarily imply greater assurability. A structural difference matters when it enables a distinction material to the assurance claim.

We initially identify seven structural dimensions that can affect this capacity:

$$D_S = \{\text{observability}, \text{controllability}, \text{traceability}, \text{reconstructability}, \\ \text{independence}, \text{integrity}, \text{evidence coverage}\}. \tag{4}$$

These dimensions are provisional. We do not claim that they are mutually independent, universally required, jointly sufficient, or complete.

Instead, the paper adopts a weaker criterion.

A structural dimension is assurance-relevant when there exists an assurance context and a pair of otherwise suitably controlled systems for which variation in that dimen-

sion creates a material difference in the assurance-relevant distinctions supported by obtainable evidence.

We refer to such a pair as a *separating construction*.

The resulting logical claim is therefore not

$$\text{all seven dimensions} \Rightarrow \text{sufficient assurance.} \tag{5}$$

Rather, the criterion for establishing a candidate dimension d requires a specified assurance context and a controlled pair of systems exhibiting both a difference in the dimension and an independently justified difference in claim-material evidentiary capabilities.

In this sense, omitting d from a general model can collapse systems that differ materially in assurability for at least one assurance context.

The framework is consequently open-ended. If a future architecture, threat model, assurance claim, or measurement mechanism reveals an additional structural property that admits a separating construction not representable by the current dimensions, that property can be added without invalidating the existing framework.

The paper also avoids defining a universal scalar assurability score.

Different systems may expose different assurance-relevant capabilities without either capability set containing the other. An architecture with strong internal-state observability but weak evidence independence may be incomparable to an architecture with limited internal visibility but independently generated, tamper-resistant records of consequential external actions.

The appropriate comparative structure is therefore generally a partial relation admitting equivalence, dominance, and incomparability rather than a total ranking.

This paper makes the following contributions:

1. It distinguishes *structural assurability* from property satisfaction, assurance arguments, confidence in assurance arguments, auditability, monitorability, and related concepts.
2. It defines assurability as a claim-relative property under explicit access, cooperation, and resource constraints.

3. It separates structural properties of an evidence-producing architecture from downstream evidence quality, measurement validity, uncertainty, and assurance reasoning.
4. It identifies an initial set of seven candidate structural dimensions: observability, controllability, traceability, reconstructability, independence, integrity, and evidence coverage.
5. It introduces a separating-construction criterion for demonstrating that a proposed structural dimension contributes to assurability without claiming that the dimension is universally necessary or that the resulting dimension set is sufficient.
6. It develops a comparative model in which systems are evaluated according to the assurance-relevant distinctions made possible by practically obtainable evidence, allowing equivalence, strict dominance, and incomparability without requiring a universal scalar score.
7. It applies the framework to representative AI architectures and demonstrates that richer instrumentation can increase assurability for one claim, provide no additional assurability for another, or create tradeoffs that prevent a total ordering.

The constituent ideas developed here are not individually new. Observability, controllability, testability, auditability, monitorability, assurance cases, provenance, evidence integrity, independence, and evaluation rigor each have substantial intellectual histories.

The proposed contribution is their integration around a different object of study: the structural capacity of an AI system and its evidence-producing environment to make claim-relevant distinctions possible.

The remainder of the paper proceeds as follows. Section 2 examines the foundations of the framework across assurance theory, design for testability, software architecture, safety engineering, security evaluation, auditability, AI monitoring, interpretability, measurement science, and related work. Section 3 defines structural assurability and develops the distinction among structural properties, evidence properties, and assurance constraints. Section 4 formalizes comparative assurability and establishes the separating-construction criterion. Section 5 applies the framework to representative AI architectures. Section 6 records what is established in the Lean formalization. Section 7 describes an illustrative application to an external evaluation framework. Sections 8 and 9 discuss implications, limitations, and threats to validity. Section 10

concludes.

2 Prior and Related Work

The concept of *assurability* developed in this paper does not arise from an absence of assurance research. On the contrary, substantial bodies of work address assurance cases, testability, observability, auditability, monitorability, safety engineering, software architecture, evaluation rigor, measurement validity, and evidence integrity. These traditions approach a common problem from different directions: how justified claims about a complex engineered system can be supported by evidence.

The contribution developed here is therefore integrative rather than terminological. We distinguish *assurance*, which concerns justified confidence in claims about a system, from *assurability*, which concerns the structural capacity of a system and its operational environment to make adequate assurance evidence obtainable. We further treat assurability as relative to a particular assurance claim, access and cooperation conditions, and resource constraints. This section reviews the principal intellectual foundations for that distinction.

2.1 Assurance Cases and Evidence-Based Assurance

Assurance cases provide one of the most mature foundations for reasoning about justified confidence in system properties. Structured assurance arguments connect claims about a system to supporting arguments and evidence, often using Claims–Arguments–Evidence structures or notations such as Goal Structuring Notation (GSN) Bloomfield and Bishop (2010); National Institute of Standards and Technology (2009). Related research has investigated the strength of assurance arguments, the treatment of defeaters, and methods for reasoning about confidence in assurance cases Graydon and Holloway (2017).

Recent work has extended assurance-case reasoning to machine learning and frontier AI. AMLAS provides a systematic methodology for integrating assurance activities into the machine-learning lifecycle and generating evidence supporting safety claims Hawkins et al. (2021). Subsequent work has demonstrated complete ML safety cases and investigated the relationship between ML evidence, safety claims, and causal models of assurance Borg et al. (2023); Burton (2022). Frontier-AI researchers

have similarly proposed safety-case structures connecting risk models, proxy tasks, evaluation environments, evidence, and claims Goemans et al. (2024). More recent work explores composable and machine-readable assurance assertions capable of propagating through AI development and deployment workflows Zhao (2026).

This literature establishes how evidence may support an assurance argument. Our concern is complementary: the architecture of the system itself affects which evidence can be obtained, how independently it can be obtained, how much it costs to obtain, and which assurance claims can therefore be supported in practice. An assurance case describes an argument from evidence; assurability characterizes structural conditions affecting the availability and quality of that evidence.

2.2 Design for Testability and System Observability

Hardware design for testability (DfT) provides a particularly important precedent. As integrated circuits became too complex to test economically solely through external interfaces, designers introduced architectural mechanisms specifically intended to improve internal *controllability* and *observability* Williams and Parker (1982); Abramovici et al. (1990); IEEE (2026). Test points, scan structures, built-in self-test, and related mechanisms make faults easier to activate and their effects easier to observe.

The central DfT insight is architectural: testability is not merely a property of the test procedure. The design of the artifact affects the feasibility, completeness, and cost of testing it. Consequently, systems exhibiting equivalent functional behavior may differ substantially in the effort required to establish confidence in their correct operation.

Software-engineering research generalizes related ideas through software testability, controllability, observability, diagnosability, and analyzability Binder (1994); Freedman (1991). These properties affect whether test conditions can be established, whether relevant internal or external states can be observed, and whether failures can be diagnosed efficiently.

We adopt the underlying structural insight while extending its scope. Testing is one mechanism for producing assurance evidence, but assurance may additionally depend upon runtime monitoring, provenance, audit records, independent observation, internal-state measurements, environmental evidence, and post-incident

reconstruction. Assurability therefore generalizes the design-for-testability intuition from the feasibility of testing to the feasibility of producing evidence for specified assurance claims.

2.3 Software Architecture and Quality Attributes

Software architecture research provides a second important foundation. Quality attributes such as performance, availability, security, modifiability, testability, and analyzability are influenced by architectural decisions and frequently interact through tradeoffs Bass et al. (2021). The Architecture Tradeoff Analysis Method (ATAM) evaluates architectures using concrete quality-attribute scenarios and identifies risks, sensitivity points, and tradeoff points rather than assuming a universally optimal architecture Clements et al. (2002); Kazman et al. (2000).

Recent work on quality models for machine-learning components explicitly includes analyzability, monitorability, and testability among relevant architectural properties Lewis et al. (2026). These attributes concern the extent to which a component exposes information permitting efficient behavioral assessment, runtime measurement, testing, and diagnosis.

This scenario-based view motivates our treatment of assurability as claim-relative rather than as a single context-independent score. An architecture may be highly assurable with respect to one claim while providing little useful evidence for another. Similarly, architectures may trade internal observability against evidence independence, integrity, cost, performance, or information disclosure. Comparative assurability should therefore permit partial ordering and incomparability rather than require a universal scalar ranking.

2.4 Safety Engineering and Development Assurance

Safety-critical engineering has developed extensive methods for connecting system hazards, development processes, architectural decisions, evidence, and required assurance.

Aviation standards such as ARP4754A and DO-178C vary development-assurance rigor according to the consequences of failure and impose increasingly rigorous verification activities for safety-critical functions SAE International (2010); RTCA

(2011).

Independence is itself an assurance-relevant architectural and process attribute: ARP4754A uses independence in development-assurance-level allocation and applies independence requirements to activities including requirements validation, implementation verification, and process assurance SAE International (2010).

More generally, safety engineering employs hazard analysis, fault analysis, independent verification and validation, defense in depth, runtime assurance, and structured safety cases to establish confidence in safety-relevant claims Leveson (2011); Rushby (2007).

These traditions establish an important distinction for the present work. The *required level of assurance* may be determined by the consequence associated with failure of a claim, while the *assurability* of the architecture determines how readily the evidence necessary to satisfy that requirement can be obtained. A high assurance requirement does not imply that the system architecture makes the required assurance practically achievable.

2.5 Security Evaluation and the Common Criteria

The Common Criteria for Information Technology Security Evaluation provides a mature framework connecting security claims, evidence requirements, evaluation rigor, and evaluation feasibility Common Criteria Recognition Arrangement (2022a). Evaluation Assurance Levels (EALs) provide predefined packages of progressively rigorous assurance requirements and explicitly balance the assurance sought against the cost and feasibility of obtaining that assurance Common Criteria Recognition Arrangement (2022b).

The Common Criteria therefore provides important precedent for treating assurance as constrained by practical resource considerations. It also supports property-specific evaluation and equivalence reasoning rather than treating security as an undifferentiated system property.

Evaluation rigor, however, is conceptually distinct from structural assurability. An EAL characterizes the depth and rigor of evaluation applied to specified security claims. It does not, by itself, characterize how effectively the architecture exposes trustworthy evidence relevant to those claims. Two systems subjected to equivalent

evaluation rigor may differ substantially in the effort required to produce equivalent evidence.

2.6 Auditability and Accountability

Auditability research addresses the accessibility and preservation of evidence needed for retrospective or independent examination. Domain-specific work demonstrates that architecture can strongly influence auditability. NIST analyses of voting systems, for example, compare alternative architectures according to their ability to provide independent and trustworthy evidence supporting election audits National Institute of Standards and Technology (2003). The comparison demonstrates that systems performing nominally equivalent functions can differ materially in the evidence their architectures make available.

Recent AI research similarly argues for auditability-by-design. Proposed frameworks connect verifiable claims to the evidence necessary to evaluate those claims and examine whether AI systems make that evidence technically accessible Fernsel et al. (2024). Practitioner-oriented work identifies auditability mechanisms spanning AI design, development, deployment, and operation Li and Goel (2025), while continuous-auditing architectures attempt to preserve evidence throughout the AI lifecycle Waltersdorfer et al. (2024).

A recent taxonomy of opaque, explainable, and replayable AI architectures further argues that accountability capabilities depend upon whether evidence is absent, reconstructed after execution, or preserved as part of the system architecture Akkinapragada (2026). This work is especially close to the present argument because it treats evidence availability as an architectural property.

Auditability nevertheless addresses a narrower question than the assurability construct developed here. Assurance may require evidence that is observable during execution, experimentally elicitable, independently generated, internally measured, calibrated, or causally validated in addition to evidence preserved for subsequent audit.

2.7 AI Monitorability and Internal-State Evidence

AI monitorability research examines whether properties of an AI system or agent can be detected from available observation channels. Formal treatments have considered limits on the ability to observe and predict AI behavior Yampolskiy (2025). Recent frontier-model research treats monitorability as a property of the relationship between an agent, an observation channel, and a monitor, and evaluates information available from actions, outputs, chains of thought, and internal activations Guan et al. (2025).

This literature becomes increasingly important as evaluation awareness creates a possible divergence between behavior observed under evaluation and behavior that would occur under deployment conditions. Work on scheming, evaluation awareness, and loss-of-control evaluations has therefore motivated deeper evaluator access to model internals and white-box evaluation methods Apollo Research (2026); Meinke et al. (2024).

Mechanistic interpretability contributes potential measurement channels into learned computation itself, including activation analysis, representation probing, sparse feature extraction, attribution, circuit analysis, and causal intervention Olah et al. (2020); Elhage et al. (2022); Bricken et al. (2023). Such methods raise the possibility that internal-state evidence could complement behavioral and runtime evidence.

However, observing an activation or representation is not equivalent to observing a latent assurance construct such as deception, intent, evaluation awareness, or policy compliance. Internal-state measurements therefore introduce measurement-validity and calibration problems in addition to observability problems.

2.8 Agent Observability and Runtime Assurance

Agentic AI extends the assurance problem beyond model inputs and outputs. Agent behavior may depend upon tool calls, retrieved information, persistent memory, authorization state, environmental observations, external services, human interventions, and long-running execution trajectories.

Agent observability work is increasingly standardizing telemetry for these interactions. OpenTelemetry and related initiatives are extending distributed tracing concepts to AI-agent execution, including model calls, tool use, and agent trajectories

OpenTelemetry (2025). Security-oriented efforts similarly seek standardized agent observability records capable of supporting monitoring and investigation OWASP (n.d.).

Runtime-assurance research addresses the complementary problem of monitoring and constraining behavior while a system operates. Runtime monitoring and agent observability provide mechanisms for recording execution events and examining behavior during operation Rushby (2007); OpenTelemetry (2025); OWASP (n.d.). NIST's work on evaluation probes for agentic AI similarly investigates embedding adversarial verification mechanisms within agent workflows and producing machine-readable audit evidence National Institute of Standards and Technology (2026b).

These approaches provide important components of an assurance evidence architecture. They do not, individually, establish how evidence from heterogeneous observation planes should be compared or how architectural alternatives should be evaluated according to their overall capacity to support particular assurance claims.

2.9 Measurement Science, Validity, and Uncertainty

Assurance depends not merely upon collecting observations but upon establishing that measurements support the constructs and claims attributed to them. Measurement science therefore provides another essential foundation. Calibration, measurement uncertainty, traceability, construct validity, criterion validity, and reproducibility constrain the strength of conclusions that may legitimately be drawn from observed data Joint Committee for Guides in Metrology (2008); Messick (1995).

This distinction is particularly important for AI assurance because many properties of interest are latent constructs. Correctness may sometimes be externally established, but properties such as deceptive intent, evaluation awareness, strategic planning, or alignment are generally inferred from observable proxies.

The distinction between observation and inference motivates an explicit measurement chain:

$$\begin{aligned} \text{construct} &\rightarrow \text{observable} \rightarrow \text{measurement} \\ &\rightarrow \text{inference} \rightarrow \text{evidence} \rightarrow \text{assurance claim.} \end{aligned} \tag{6}$$

Each transition introduces assumptions that may limit the resulting assurance claim.

A highly instrumented system is therefore not necessarily highly assurable if the relationship between its measurements and the relevant assurance construct cannot be validated.

2.10 Adversarial Monitoring, Evidence Integrity, and Noncooperation

Cybersecurity contributes a different assumption that is essential for AI assurance: the system or actor being evaluated may not cooperate. Security engineering therefore emphasizes privilege separation, independent monitoring, tamper-evident records, trusted computing bases, adversarial detection, fault containment, and defense in depth Anderson (2020); Saltzer and Schroeder (1975).

This differs materially from assurance approaches that assume evaluators receive trustworthy telemetry, complete system access, or accurate developer disclosures. AI assurance methods must therefore state their dependence upon cooperation and distinguish evidence available under cooperative, partially cooperative, and adversarial access conditions.

Evidence integrity is particularly important. A system that generates extensive telemetry but can selectively suppress or retrospectively alter that telemetry may provide weaker assurance evidence than a less observable system whose smaller evidence stream is independently protected. Thus, observability alone cannot determine assurability.

2.11 Economics and the Cost of Assurance

Several mature assurance disciplines recognize that assurance is constrained by cost. Hardware DfT arose partly because exhaustive external testing became economically infeasible as system complexity increased. Common Criteria explicitly relates evaluation assurance to the cost and feasibility of obtaining it Common Criteria Recognition Arrangement (2022b). Software architecture methods similarly reason about tradeoffs among quality attributes and engineering resources Bass et al. (2021).

Current AI monitoring work identifies computational cost, organizational burden, scalability, and competitive pressure as practical barriers to continuous assurance

National Institute of Standards and Technology (2026a). These considerations become especially important when compliant systems compete against systems that omit costly safeguards.

We therefore treat assurance cost as a first-class constraint rather than an implementation detail. A theoretically strong assurance mechanism that imposes prohibitive compute, latency, capability, development, disclosure, or verification costs may provide limited practical assurance at scale.

This motivates comparison of assurance architectures under explicit resource constraints and suggests an *assurance frontier*: the set of architectures for which assurance capability cannot be improved along relevant dimensions without increasing cost or sacrificing another relevant property.

2.12 From Fragmented Properties to Comparative Assurability

The preceding literatures provide substantial portions of the required conceptual machinery. Hardware engineering provides controllability and observability. Software architecture provides quality attributes, scenarios, and tradeoff analysis. Safety engineering provides claim-criticality relationships, independence, and structured assurance. Security evaluation provides evaluation rigor and explicit treatment of assurance cost. Auditability provides evidence accessibility and preservation. Monitorability provides observation-channel analysis. Runtime assurance and agent observability provide execution evidence. Mechanistic interpretability provides potential access to learned internal state. Measurement science provides validity, calibration, and uncertainty. Cybersecurity provides adversarial assumptions, evidence integrity, and noncooperation.

What remains less clearly formalized is the common structural property connecting these concepts.

We use *assurability* to denote the capacity of a system and its evidence-producing environment to support or refute a specified assurance claim through valid, sufficient, appropriately independent, and practically obtainable evidence.

Crucially, assurability is not treated as an unconditional scalar property of a system. We provisionally represent it as claim-, access-, and resource-relative:

$$\mathcal{A}(S, C, K, B), \tag{7}$$

where S denotes the system, C the assurance claim, K the knowledge, access, and cooperation conditions available to the evaluator, and B the resource constraints under which assurance must be obtained.

This framing separates four concepts that are frequently adjacent but not equivalent:

$$\begin{aligned} &\text{system property} \neq \text{assurability} \\ &\qquad \neq \text{assurance argument} \neq \text{confidence in the argument.} \end{aligned} \tag{8}$$

It also permits comparative reasoning without assuming that heterogeneous assurance properties can be reduced defensibly to a universal numerical score. For two systems S_1 and S_2, a relation of the form

$$S_1 \succeq_{C,K,B} S_2 \tag{9}$$

can express that S_1 is at least as assurable as S_2 with respect to claim C, under conditions K, within resource constraints B. Such a relation may induce only a partial order: competing architectures may be incomparable when they expose different assurance capabilities or trade evidence strength against cost, independence, integrity, or other dimensions.

The proposed framework therefore does not replace existing assurance, auditability, monitorability, interpretability, or evaluation methodologies. It attempts to provide an architectural abstraction through which their contributions can be related and competing AI systems can be compared according to the evidence they make practically available for assurance.

3 Problem Setting

Assurance is fundamentally concerned with claims and evidence. Existing assurance theory provides substantial machinery for reasoning from evidence to claims: whether evidence is relevant and sufficient, whether an assurance argument is sound, what

assumptions it depends upon, what defeaters remain, and what confidence in the resulting claim is justified.

This work considers an antecedent question:

> What structural properties determine whether evidence relevant to supporting or refuting an assurance claim can be obtained in the first place?

This question is distinct from whether the claim is true and from whether a particular assurance argument is convincing. A system may satisfy a desired property while exposing little evidence that it does so. Conversely, a system may expose extensive and reliable evidence demonstrating that a desired property is not satisfied.

We use the term *assurability* for the capacity to make assurance-relevant evidence obtainable. We use *structural assurability* when emphasizing architectural and environmental properties that contribute to this capacity.

The objective of this section is deliberately limited. We define the object of study, identify the context in which assurability must be evaluated, distinguish structural properties from evidence-quality properties and practical constraints, and identify candidate structural dimensions whose contribution can be demonstrated through separating constructions.

We do not claim that these dimensions are jointly sufficient to characterize assurability, individually necessary for every assurance problem, mutually independent, or complete.

3.1 From Assurance Evidence to Evidence Availability

Let S denote a system and C an assurance claim concerning S. Let E denote evidence relevant to C, and let G denote an assurance argument connecting that evidence, together with applicable assumptions, to the claim.

At a high level,

$$E \xrightarrow{G} C. \tag{10}$$

Much assurance research appropriately concentrates on this relationship: whether E supports C, whether G is sound, what counterevidence or defeaters exist, and what confidence is justified.

Before such reasoning can occur, however, relevant evidence must be obtainable.

Consider systems S_1 and S_2 that implement the same externally visible functionality and exhibit equivalent behavior over an evaluation set T:

$$\text{Behavior}_T(S_1) = \text{Behavior}_T(S_2). \tag{11}$$

Suppose S_1 exposes only externally visible inputs and outputs, while S_2 additionally preserves authenticated records of consequential actions, tool invocations, retrieved information, authorization decisions, provenance, and execution state.

Behavioral equivalence does not imply evidentiary equivalence. The systems may therefore be indistinguishable with respect to T while differing materially in what can subsequently be established about their operation.

This observation has mature analogues in other engineering disciplines. Design for testability modifies system architecture to improve the ability to establish test conditions and observe relevant system behavior. Software architecture similarly affects testability, analyzability, monitorability, and the cost and effectiveness of verification activities Abramovici et al. (1990); Bass et al. (2021).

The corresponding structural principle for assurance is:

> The architecture of a system and its evidence-producing environment constrain the evidence that can practically be obtained concerning assurance claims about that system.

3.2 Definition of Assurability

The preceding observation motivates a deliberately minimal definition.

Definition 1 (Assurability)**.** The *assurability* of a system S with respect to an assurance claim C, under conditions K and resource constraints B, is the capacity of the system and its evidence-producing environment to make evidence relevant to supporting or refuting C practically obtainable.

We denote this claim-relative capacity by

$$\mathcal{A}(S, C, K, B). \tag{12}$$

The notation does not imply that $\mathcal{A}$ is scalar-valued. Indeed, the framework developed here deliberately avoids assuming that heterogeneous assurance properties can be reduced to a meaningful universal score.

Several features of Definition 1 are intentional.

First, assurability concerns evidence, not truth. A highly assurable system may provide decisive evidence that claim C is false.

Second, assurability concerns evidence capable of supporting *or refuting* C. A structure that facilitates favorable evidence while systematically obscuring disconfirming evidence should not thereby be considered highly assurable.

Third, the definition does not specify which evidence is sufficient for establishing C. Sufficiency belongs to the downstream assurance problem and depends upon the claim, argument, evidence model, threat model, and applicable assurance requirements.

Fourth, the definition does not require that evidence originate within the system. The evidence-producing environment may include deployment infrastructure, independent monitors, external evaluators, physical sensors, network services, human processes, or other systems.

Assurability is therefore distinct from the property being assured, from the assurance argument, and from confidence in that argument:

$$\boxed{\begin{aligned}&\text{Property satisfaction} \neq \text{Assurability}\\ &\qquad\qquad\qquad\qquad \neq \text{Assurance argument} \neq \text{Confidence in the argument.}\end{aligned}} \tag{13}$$

3.3 Assurance Context

Assurability is not an unconditional property of S.

A system may provide strong evidence concerning whether an external action occurred while providing little evidence concerning the internal computational process that produced the action. It may therefore be highly assurable with respect to one claim and weakly assurable with respect to another.

Evidence availability also depends upon evaluator access and cooperation. A developer may possess model weights, activations, system prompts, execution traces, configuration data, and infrastructure logs. An external evaluator may possess only API access. An independent assessor may receive selected evidence but lack direct system access. In an adversarial setting, the developer, operator, or system itself may attempt to suppress, manipulate, or fabricate evidence.

We collect these knowledge, access, trust, and cooperation conditions in K.

Finally, theoretically obtainable evidence may not be practically obtainable. Assurance consumes computation, time, expertise, storage, instrumentation, system access, and other resources. Assurance mechanisms may additionally impose latency, performance, disclosure, or operational costs. We represent relevant resource constraints by B.

We therefore define:

Definition 2 (Assurance Context)**.** An *assurance context* is a tuple

$$Q = (S, C, K, B), \tag{14}$$

specifying the system under consideration, the assurance claim, the applicable knowledge, access, trust, and cooperation conditions, and the resource constraints under which assurance evidence must be obtained.

Comparative statements about assurability are meaningful only relative to a sufficiently specified assurance context.

In particular,

> System S_1 is more assurable than system S_2

is generally underspecified.

The comparison must identify what is being assured and under what conditions.

3.4 Structure, Evidence Quality, and Constraints

A useful characterization of assurability requires distinguishing three classes of factors that are easily conflated:

1. structural properties of the system and evidence-producing environment;
2. properties governing the evidentiary or inferential quality of observations; and
3. contextual constraints governing whether evidence can practically be obtained.

These occupy different roles in the assurance process.

Structural properties. Structural properties determine what can be exposed, controlled, associated with its origin, preserved, reconstructed, independently corroborated, protected, and covered by the evidence-producing architecture. These properties are the primary subject of structural assurability.

Evidence and inference properties. Once evidence is available, additional questions arise concerning its relevance, validity, fidelity, calibration, and uncertainty. These properties govern what may legitimately be inferred from an observation or measurement.

For example, an architecture may expose an internal measurement perfectly while the measurement itself is an invalid indicator of the construct it is intended to represent. Greater structural observability does not resolve construct validity.

Contextual constraints. Access, cooperation, time, computation, expertise, financial cost, disclosure restrictions, and operational burden constrain which structurally available evidence can actually be obtained in a particular assurance context. These constraints are represented principally through K and B.

The distinction can be summarized as

$$\boxed{\begin{aligned}&\text{Structure} \longrightarrow \text{Obtainable Evidence} \\ &\qquad\qquad \longrightarrow \text{Evidence and Inference} \longrightarrow \text{Assurance Argument},\end{aligned}} \tag{15}$$

subject throughout to the conditions and constraints of the assurance context.

This separation is important because the present theory concerns the first transition. Structural assurability constrains what evidence can be made available; it does not by itself establish the validity of subsequent inference or the sufficiency of an assurance argument.

3.5 Candidate Structural Dimensions

Prior work across hardware testing, software architecture, safety engineering, security evaluation, auditability, runtime assurance, and AI monitoring suggests several recurring structural properties that affect the availability and usefulness of assurance evidence.

We begin with seven candidate dimensions:

Observability. The extent to which states, events, behaviors, or effects relevant to an assurance claim can be observed.

Controllability. The extent to which conditions relevant to evaluating an assurance claim can be established and relevant interventions can be performed.

Traceability. The extent to which evidence can be associated with its origin, transformation history, and relevant system events.

Reconstructability. The extent to which a relevant execution, decision sequence, or system state can subsequently be reconstructed from preserved evidence.

Independence. The extent to which evidence can be generated, preserved, or corroborated independently of the component or actor whose behavior is the subject of the assurance claim.

Integrity. The extent to which alteration, fabrication, suppression, substitution, or corruption of assurance evidence can be detected or prevented.

Evidence Coverage. The extent to which the evidence-producing structure exposes the states, behaviors, events, interfaces, or effects material to the assurance claim.

We denote the current candidate set by

$$D_S = \{O, C, T, R, N, I, E\}, \tag{16}$$

where the symbols are used only as compact identifiers and do not imply numerical measurement scales.

The candidate set is deliberately provisional.

The dimensions are not asserted to be mutually independent. They may interact or partially depend upon one another. Reconstructability, for example, may depend upon observability, traceability, and evidence integrity. Evidence coverage may depend upon the placement and scope of observation mechanisms.

Nor is each dimension asserted to be necessary for every assurance claim. Some claims require no access to internal state. Some require no intervention. Some can be established through independently observable external effects.

The purpose of the candidate dimensions is therefore not to define a checklist that every assurable system must satisfy. It is to identify structural variables whose omission can erase distinctions that matter to assurance.

3.6 Derived Assurance Capabilities

Several important properties associated with assurance appear better treated as capabilities emerging from combinations of structural dimensions rather than as primitive dimensions themselves.

These include, potentially,

$$\{\text{reproducibility}, \text{auditability}, \text{monitorability}, \text{testability}, \text{diagnosability}\}. \tag{17}$$

For example, reproducibility may depend upon sufficient traceability, reconstructability, controllability, configuration preservation, and environmental information. Testability classically depends in substantial part upon observability and controllability.

Treating these properties as derived rather than primitive avoids assuming independence where the underlying engineering relationships suggest composition.

This classification is itself provisional. If a proposed derived capability contains an assurance-relevant structural property not represented by the current primitive set, that property should be identified and tested separately.

3.7 The Dimension Contribution Criterion

A candidate structural dimension should not be included merely because it appears intuitively related to assurance. Its contribution should be demonstrable.

We therefore introduce a criterion based on separating constructions.

Definition 3 (Assurance-Relevant Structural Dimension)**.** A candidate structural dimension d is *assurance-relevant* if there exists an assurance context and a pair of systems for which a difference in d, while holding other properties material to the comparison fixed, produces a material difference in the evidence practically obtainable for supporting or refuting the assurance claim.

Let S_d^+ and S_d^- denote systems constructed to differ materially with respect to candidate dimension d while remaining equivalent with respect to other properties material to a specified comparison.

Let $\mathcal{E}(S, C, K, B)$ denote the claim-relevant evidence practically obtainable from S under conditions K and constraints B.

Dimension d is assurance-relevant if there exist C, K, and B such that

$$\mathcal{E}(S_d^+, C, K, B) \not\equiv_C \mathcal{E}(S_d^-, C, K, B), \tag{18}$$

where $\not\equiv_C$ denotes a difference in obtainable evidence material to supporting or refuting C.

We call such a pair a *separating construction* for d.

The logical strength of this criterion is intentionally limited.

A separating construction does not establish that d is required for every assurance context. It does not establish that increasing d monotonically increases assurability. It does not establish that d is independent of other dimensions.

It establishes only that

> there exists an assurance context in which omitting d from the model would collapse a distinction between systems that materially affects their capacity to support or refute an assurance claim.

This is the sense in which a dimension is necessary to the general framework: not because every assurable system must possess it, but because a framework incapable of representing it would fail to distinguish some systems that differ materially in assurability.

3.8 Separating the Candidate Dimensions

The contribution criterion can be illustrated informally for each current candidate dimension.

Observability. Consider otherwise equivalent systems in which only one exposes a state relevant to C. If direct observation of that state provides evidence unavailable from the other system, observability separates their assurability.

Controllability. Consider otherwise equivalent observable systems in which only one permits the evaluator to establish a condition or intervention required to investigate C. The ability to obtain evidence under that condition separates their assurability.

Traceability. Consider systems that expose the same event but differ in whether the event can be reliably associated with its originating component, action, or causal history. For a claim whose evaluation depends upon attribution, traceability separates their assurability.

Reconstructability. Consider systems with equivalent runtime behavior and live observability, but only one preserves sufficient state and history to reconstruct a relevant execution after an incident. For retrospective assurance claims, reconstructability separates their assurability.

Independence. Consider systems providing equivalent records, but in one system the only record is generated by the component being evaluated, while the other additionally provides independently generated or protected corroboration. For claims vulnerable to self-reporting or common-mode failure, independence separates their assurability.

Integrity. Consider systems exposing equivalent records, but only one prevents or detects alteration, fabrication, or suppression of those records. For claims relying upon the historical record, integrity separates their assurability.

Evidence Coverage. Consider systems whose individual observations are otherwise equivalent, but one exposes only a proper subset of the claim-relevant action surface. For claims spanning the omitted behavior, evidence coverage separates their assurability. These examples motivate the dimension-specific constructions examined in Section 4.

Section 4 develops the comparative relation required to state these results more precisely.

3.9 Contribution Without Sufficiency

The framework deliberately distinguishes between demonstrating that a dimension contributes to assurability and claiming that a collection of dimensions completely characterizes assurability.

For a candidate dimension d, a separating construction can demonstrate that d cannot generally be discarded without losing an assurance-relevant distinction.

It does not establish that d is necessary for every assurance claim.

Likewise, establishing the relevance of every member of the current candidate set D_S does not establish that D_S is sufficient or complete.

In particular, we do not claim

$$\bigwedge_{d_i \in D_S} d_i \implies \text{sufficient assurance}, \tag{19}$$

nor do we claim that D_S exhausts the structural properties relevant to future assurance problems.

A new system architecture, assurance claim, threat model, measurement mechanism, or evidence source may expose an additional structural property d_{n+1}.

If that property admits a separating construction under Definition 3, the framework

can be extended:

$$D'_S = D_S \cup \{d_{n+1}\}. \tag{20}$$

This open-endedness is intentional.

The space of future assurance claims, architectures, threats, measurements, and evidence mechanisms is not closed. A claim that the current dimensions are jointly sufficient would therefore require assumptions that this work cannot justify.

The contribution is narrower:

> We identify structural dimensions that demonstrably affect assurability, provide a criterion for establishing their contribution, and construct a framework that can be extended when additional assurance-relevant dimensions are identified.

3.10 Structural Neutrality

Structural assurability does not determine the outcome of assurance.

An evidence-producing architecture should support examination regardless of whether the resulting evidence confirms or defeats the assurance claim. The structure should not embed the favorable conclusion it is intended to help evaluate.

Thus,

$$\text{greater structural capacity for assurance} \not\Rightarrow \text{a more favorable assurance outcome.} \tag{21}$$

Indeed, a highly assurable system may be easier to reject precisely because its structure makes adverse evidence difficult to conceal.

This neutrality separates evidence availability from downstream interpretation and judgment. Structural assurability concerns the conditions under which relevant evidence can be obtained and examined; it does not determine what conclusion an evaluator must draw from that evidence.

3.11 The Comparative Problem

The preceding definitions lead to the central problem of comparative assurability.

Consider systems S_1 and S_2 subject to the same assurance claim C, knowledge and access conditions K, and resource constraints B.

The systems may exhibit equivalent functional behavior while differing along one or more assurance-relevant structural dimensions.

The comparative question is:

> Do structural differences between S_1 and S_2 create material differences in the evidence practically obtainable for supporting or refuting C?

Definition 4 (Evidentiary Equivalence)**.** Two systems S_1 and S_2 are *evidentially equivalent* relative to a claim C, conditions K, and constraints B if their practically obtainable evidence has no difference material to supporting or refuting C. We write

$$\mathcal{E}(S_1, C, K, B) \equiv_C \mathcal{E}(S_2, C, K, B).$$

Definition 5 (Comparative Assurability)**.** *Comparative assurability* evaluates the relative capacity of two systems to make evidence relevant to supporting or refuting the same claim practically obtainable under specified, comparable conditions and resource constraints. The comparison may yield equivalence, dominance, tradeoffs, or incomparability; it does not presuppose a scalar score or total order.

Behavioral equivalence does not imply assurability equivalence.

This motivates the central premise of the framework:

> **Central Premise.**
> Systems with equivalent functional behavior or measured performance can differ materially in assurability because their architectures and evidence-producing environments can differ in assurance-relevant structural dimensions.

The contribution is therefore neither a universal assurance score nor a finite checklist whose satisfaction guarantees assurance.

It is a framework for identifying, demonstrating, and comparing structural factors that affect the practical capacity to obtain assurance evidence.

Section 4 develops the comparative relation formally and examines the candidate dimensions through separating constructions, while permitting equivalence, dominance, tradeoffs, and incomparability rather than requiring a total ordering.

4 A Formal Model of Comparative Assurability

Section 3 defined assurability as the capacity of a system and its evidence-producing environment to make evidence relevant to supporting or refuting an assurance claim practically obtainable. It further identified candidate structural dimensions and proposed a separating-construction criterion for determining whether a dimension contributes to assurability.

This section develops that framework formally.

The objective is deliberately limited. We do not define a universal scalar measure of assurability, assert that the current structural dimensions are complete, or claim that any combination of them is sufficient for assurance. Instead, we define a comparative model capable of representing when two systems differ in the assurance-relevant evidence capabilities made possible by their structures.

The model permits equivalence, strict dominance, tradeoffs, and incomparability without requiring heterogeneous assurance properties to be collapsed into a single score.

4.1 Comparative Assurance Context

Recall that an assurance context includes a system S, an assurance claim C, applicable knowledge, access, trust, and cooperation conditions K, and resource constraints B.

For comparison between systems, we hold the non-system components fixed and define

$$q = (C, K, B) \tag{22}$$

as a *comparative assurance context*.

Systems S_1 and S_2 are therefore compared under the same claim, access and cooperation assumptions, and resource constraints.

Let

$$\mathcal{E}_q(S) \tag{23}$$

denote the evidence practically obtainable from or about system S under q.

The term *practically obtainable* is essential. Evidence that could be recovered only by violating the access assumptions in K, exceeding the resource constraints in B, relying upon unavailable cooperation, or introducing capabilities not present in the specified evidence-producing environment is not included in $\mathcal{E}_q(S)$.

Evidence need not originate within S. It may be produced by deployment infrastructure, independent monitors, physical observations, human processes, external evaluators, or other components permitted by K.

4.2 Claim-Material Evidence

Not every observable fact about a system contributes to every assurance problem.

For example, exposing additional internal telemetry does not improve assurability for a claim if that telemetry cannot legitimately affect support for or refutation of the claim.

We therefore distinguish evidence availability from claim materiality.

Let

$$\mathrm{Mat}_C(e) \tag{24}$$

denote that evidence item e is material to claim C: that is, e can legitimately affect an assurance argument concerning C under the applicable evidentiary and inferential standards.

The claim-material evidence set is

$$\mathcal{E}_q^C(S) = \{e \in \mathcal{E}_q(S) \mid \mathrm{Mat}_C(e)\}\,. \tag{25}$$

Materiality is not itself treated as a structural property. Whether an observation legitimately bears on a claim is an evidentiary and inferential question.

Structural assurability concerns whether potentially material evidence can be made obtainable. It does not guarantee that an evaluator will interpret that evidence validly.

4.3 Evidentiary Capabilities

Direct comparison of raw evidence sets is insufficient.

Two systems may produce different artifacts while enabling the same assurance-relevant inquiry. Conversely, two systems may expose superficially similar records while differing substantially in what those records permit an evaluator to establish.

We therefore compare systems through the *evidentiary capabilities* enabled by their evidence.

Let

$$\mathfrak{G}_C \tag{26}$$

denote the universe of evidentiary capabilities potentially material to claim C.

An evidentiary capability is an assurance-relevant investigative or discriminative capability enabled by available evidence. Examples include the ability to:

- determine whether a specified event occurred;
- distinguish among candidate sources of an event;
- associate an action with an authorization decision;
- reconstruct a relevant execution path;
- determine whether a specified state was reached;
- corroborate a self-generated record independently;
- detect alteration or suppression of a historical record;

- examine behavior under a specified intervention or condition; or
- exclude a claim-relevant class of otherwise plausible system behaviors.

Let

$$\Gamma_C(E) \subseteq \mathfrak{G}_C \tag{27}$$

denote the evidentiary capabilities enabled by evidence set E, under fixed evidentiary and inferential standards.

For a system S under comparative assurance context q, define

$$\mathcal{G}_q(S) = \Gamma_C\left(\mathcal{E}_q^C(S)\right). \tag{28}$$

Thus, $\mathcal{G}_q(S)$ represents not merely the evidence artifacts obtainable from S, but the claim-material assurance inquiries those artifacts enable.

This abstraction separates three questions:

$$\boxed{\begin{array}{rcl} \mathcal{E}_q(S) & : & \text{What evidence can be obtained?} \\ \mathcal{E}_q^C(S) & : & \text{What obtainable evidence is material to } C\text{?} \\ \mathcal{G}_q(S) & : & \text{What assurance-relevant capabilities does that evidence enable?} \end{array}} \tag{29}$$

The first question is principally structural and contextual. The second and third depend additionally upon the applicable evidence and inference model.

For comparative structural analysis, those downstream standards are held fixed.

4.4 An Evidence-Capability Preorder

The capability representation permits a more precise comparison than the informal phrase "provides no less capability."

For claim C, define a relation on evidence sets by

$$E_1 \preceq_C^E E_2 \quad \iff \quad \Gamma_C(E_1) \subseteq \Gamma_C(E_2). \tag{30}$$

Thus, E_2 is at least as capable as E_1 with respect to claim C when every assurance-relevant capability enabled by E_1 is also enabled by E_2.

The relation is reflexive because

$$\Gamma_C(E) \subseteq \Gamma_C(E), \tag{31}$$

and transitive because

$$\Gamma_C(E_1) \subseteq \Gamma_C(E_2) \quad \text{and} \quad \Gamma_C(E_2) \subseteq \Gamma_C(E_3) \tag{32}$$

imply

$$\Gamma_C(E_1) \subseteq \Gamma_C(E_3). \tag{33}$$

It is therefore a preorder on evidence sets.

Distinct evidence sets may occupy the same equivalence class if they enable the same claim-material capabilities.

This is desirable. The theory concerns what evidence permits an evaluator to establish, not the incidental representation of the evidence artifact.

4.5 Evidentiary and Assurability Equivalence

We can now define equivalence precisely.

> **Definition 4 (Evidentiary Equivalence).** Systems S_1 and S_2 are *evidentially equivalent* under comparative assurance context q, written
>
> $$S_1 \equiv_q^E S_2, \tag{34}$$
>
> when

$$\mathcal{G}_q(S_1) = \mathcal{G}_q(S_2). \tag{35}$$

Evidentiary equivalence does not require equality of evidence artifacts.

For example, an authenticated infrastructure log and an independently observed physical event may be different artifacts while enabling the same claim-material capability for a particular assurance problem.

Likewise, additional telemetry that enables no additional claim-material capability does not by itself distinguish systems under $\equiv_q^E$.

This prevents evidence volume from being mistaken for assurability.

4.6 Comparative Assurability

We next lift the evidence-capability relation to systems.

> **Definition 5 (Comparative Assurability).** For a fixed comparative assurance context $q = (C, K, B)$, system S_1 is *at least as assurable as* system S_2, written
>
> $$S_1 \succeq_q S_2, \tag{36}$$
>
> if
>
> $$\mathcal{G}_q(S_2) \subseteq \mathcal{G}_q(S_1). \tag{37}$$

Thus, every claim-material evidentiary capability available from S_2 must also be available from S_1.

Strict comparative assurability is defined by proper inclusion:

$$S_1 \succ_q S_2 \quad \Longleftrightarrow \quad \mathcal{G}_q(S_2) \subset \mathcal{G}_q(S_1). \tag{38}$$

Assurability equivalence occurs when

$$S_1 \sim_q S_2 \quad \iff \quad \mathcal{G}_q(S_1) = \mathcal{G}_q(S_2). \tag{39}$$

Systems are incomparable when neither capability set contains the other:

$$S_1 \parallel_q S_2 \tag{40}$$

if

$$\mathcal{G}_q(S_1) \not\subseteq \mathcal{G}_q(S_2) \tag{41}$$

and

$$\mathcal{G}_q(S_2) \not\subseteq \mathcal{G}_q(S_1). \tag{42}$$

Because set containment is reflexive and transitive, $\succeq_q$ is a preorder on systems.

Systems that enable identical evidentiary capabilities form equivalence classes under $\sim_q$. The preorder induces a partial order over those equivalence classes.

This is sufficient for comparative assurability. No total ranking is required.

4.7 Behavioral Equivalence Does Not Imply Assurability Equivalence

The first general result follows directly.

Proposition 1 (Behavioral Non-Implication)**.** *Behavioral equivalence under an evaluation set does not imply assurability equivalence.*

Construction. Let systems S_1 and S_2 produce identical externally visible behavior for every input in evaluation set T:

$$\text{Behavior}_T(S_1) = \text{Behavior}_T(S_2). \tag{43}$$

Suppose both systems perform consequential action a.

System S_1 exposes the externally visible result of a.

System S_2 exposes the same result and additionally preserves authenticated evidence associating a with its initiating component, authorization state, relevant tool invocation, and execution time.

Let claim C concern whether a was initiated through an authorized system pathway.

Under a comparative assurance context q in which the authenticated evidence is accessible within B, S_2 enables at least one claim-material capability—attribution of a to the relevant pathway—that S_1 does not.

Therefore,

$$\mathcal{G}_q(S_1) \neq \mathcal{G}_q(S_2), \tag{44}$$

and hence

$$S_1 \not\sim_q S_2. \tag{45}$$

Thus,

$$\text{Behavior}_T(S_1) = \text{Behavior}_T(S_2) \not\Rightarrow S_1 \sim_q S_2. \tag{46}$$

Behavioral equivalence therefore does not imply assurability equivalence. □

The result does not require either system to be safer, more correct, or more trustworthy. It establishes only that behavioral equivalence does not determine equivalence in the structural capacity to support assurance.

4.8 Separating Constructions

We now formalize the dimension-contribution criterion introduced in Section 3.7.

Let d be a candidate structural dimension.

A *separating construction* for d consists of systems

$$S_d^-, \qquad S_d^+, \tag{47}$$

and comparative assurance context q_d, such that the systems differ materially in d, are otherwise controlled with respect to properties material to the construction, and satisfy

$$\mathcal{G}_{q_d}(S_d^-) \neq \mathcal{G}_{q_d}(S_d^+). \tag{48}$$

Equivalently,

$$S_d^- \not\sim_{q_d} S_d^+. \tag{49}$$

This non-equivalence is sufficient to establish that d contributes to assurability.

A stronger form occurs when the construction is a *conservative extension*: every capability available from S_d^- is preserved in S_d^+, and at least one additional claim-material capability is introduced.

Then

$$\mathcal{G}_{q_d}(S_d^-) \subset \mathcal{G}_{q_d}(S_d^+), \tag{50}$$

and therefore

$$S_d^+ \succ_{q_d} S_d^-. \tag{51}$$

The distinction is important:

non-equivalence establishes contribution; proper inclusion establishes directional dominance.	(52)

The general theory requires only the weaker result.

4.9 Observability

Proposition 2 (Observability Contribution)**.** *Observability is an assurance-relevant structural dimension.*

Construction. Let S_O^- and S_O^+ execute the same computation

$$x \to h_1 \to h_2 \to a \tag{53}$$

and produce identical external action a.

In S_O^-, only x and a are observable.

In S_O^+, state h_1 is additionally exposed through an observation mechanism that does not otherwise alter the system properties material to the construction.

Let C_O be a claim for which determining whether h_1 possessed a specified property is material.

Under suitable K and B, S_O^+ enables the capability

$$g_O = \text{“observe the claim-relevant property of } h_1\text{”}, \tag{54}$$

while S_O^- does not.

Therefore,

$$\mathcal{G}_{q_O}(S_O^-) \neq \mathcal{G}_{q_O}(S_O^+). \tag{55}$$

Hence observability admits a separating construction and is assurance-relevant. □

If the added observation preserves every evidentiary capability of S_O^-, then the construction is additionally directional:

$$S_O^+ \succ_{q_O} S_O^-. \tag{56}$$

This stronger conclusion belongs to the construction, not to a universal claim that

greater observability always yields greater assurability.

4.10 Controllability

Proposition 3 (Controllability Contribution)**.** *Controllability is an assurance-relevant structural dimension.*

Construction. Let S_C^- and S_C^+ expose equivalent observations during ordinary execution.

Suppose evaluation of claim C_C requires examining system behavior under condition x^*.

The evaluator cannot establish x^* in S_C^-.

In S_C^+, an assurance interface permits the evaluator to establish x^*, replay the relevant state, substitute the required environmental condition, or otherwise perform the intervention without altering the property material to C_C.

Thus S_C^+ enables

$$g_C = \text{“obtain evidence under condition } x^*\text{”}, \tag{57}$$

while S_C^- does not.

Therefore,

$$\mathcal{G}_{q_C}(S_C^-) \neq \mathcal{G}_{q_C}(S_C^+). \tag{58}$$

Controllability is therefore assurance-relevant. □

When the intervention interface conservatively adds this capability while preserving the capabilities already available from S_C^-,

$$S_C^+ \succ_{q_C} S_C^-. \tag{59}$$

4.11 Traceability

Proposition 4 (Traceability Contribution)**.** *Traceability is an assurance-relevant structural dimension distinct from mere event observability.*

Construction. Let S_T^- and S_T^+ expose the same consequential event a.

In S_T^-, the evidence establishes that a occurred but does not reliably associate it with its originating component or causal history.

In S_T^+, authenticated provenance associates

$$\text{initiator} \to \text{decision} \to \text{authorization} \to \text{tool invocation} \to a. \tag{60}$$

Let C_T concern whether a originated from a specified component or authorized pathway.

Both systems expose occurrence of a. The separating capability is therefore not event observation.

System S_T^+ additionally enables

$$g_T = \text{“attribute } a \text{ to its relevant origin or pathway”}. \tag{61}$$

Thus,

$$\mathcal{G}_{q_T}(S_T^-) \neq \mathcal{G}_{q_T}(S_T^+). \tag{62}$$

Traceability is therefore assurance-relevant even when event observability is held fixed. □

For a conservative extension,

$$S_T^+ \succ_{q_T} S_T^-. \tag{63}$$

4.12 Reconstructability

Proposition 5 (Reconstructability Contribution)**.** *Reconstructability is an assurance-relevant structural dimension.*

Construction. Let S_R^- and S_R^+ have equivalent runtime behavior and equivalent live observability.

Following execution, S_R^- discards transient state and contextual information required to reconstruct the sequence leading to consequential action a.

System S_R^+ preserves sufficient inputs, configuration, relevant state transitions, external responses, and action history to permit such reconstruction.

Let C_R be a retrospective claim concerning the pathway by which a occurred.

During live execution, the systems may expose equivalent information. After the event, however, S_R^+ enables

$$g_R = \text{“reconstruct the claim-relevant execution leading to } a\text{”}, \tag{64}$$

while S_R^- does not.

Therefore,

$$\mathcal{G}_{q_R}(S_R^-) \neq \mathcal{G}_{q_R}(S_R^+). \tag{65}$$

Reconstructability is therefore assurance-relevant even when live observability is held fixed. □

If reconstruction evidence is added without removing other capabilities, then

$$S_R^+ \succ_{q_R} S_R^-. \tag{66}$$

4.13 Independence

Proposition 6 (Independence Contribution)**.** *Independent evidence generation or corroboration is an assurance-relevant structural dimension.*

Construction. Let S_N^- and S_N^+ provide the same system-generated record concerning action a.

In S_N^-, that record is the only evidence source.

In S_N^+, an independent component outside the control of the component under evaluation produces corroborating evidence concerning the same event.

Let C_N be a claim for which failure, deception, compromise, or error in the evaluated component is within the threat model specified by K.

The system-generated record may possess the same nominal integrity in both systems.

However, S_N^+ additionally enables

$$g_N = \text{“corroborate the claim-relevant event independently”}. \tag{67}$$

Therefore,

$$\mathcal{G}_{q_N}(S_N^-) \neq \mathcal{G}_{q_N}(S_N^+). \tag{68}$$

Independence is therefore assurance-relevant even when the nominal integrity of the original evidence artifact is held fixed. □

Where independent corroboration preserves all previously available capabilities,

$$S_N^+ \succ_{q_N} S_N^-. \tag{69}$$

4.14 Integrity

Proposition 7 (Evidence Integrity Contribution)**.** *Evidence integrity is an assurance-relevant structural dimension distinct from nominal evidence availability.*

Construction. Let S_I^- and S_I^+ generate and expose the same class of execution records.

In S_I^-, an actor included within the threat model can silently alter, fabricate, delete, substitute, or suppress those records.

In S_I^+, the evidence-producing environment prevents such modification or makes it detectably evident.

Let C_I concern whether an unauthorized action occurred during the recorded interval.

Both systems nominally expose the same record type. The difference concerns whether manipulation of that historical evidence can itself be detected.

System S_I^+ therefore enables

$$g_I = \text{“detect claim-material tampering or suppression of the record”}, \tag{70}$$

while S_I^- does not.

Hence,

$$\mathcal{G}_{q_I}(S_I^-) \neq \mathcal{G}_{q_I}(S_I^+). \tag{71}$$

Evidence integrity is therefore assurance-relevant even when nominal evidence availability is held fixed. □

If integrity protection adds this capability without removing another,

$$S_I^+ \succ_{q_I} S_I^-. \tag{72}$$

4.15 Evidence Coverage

Proposition 8 (Evidence Coverage Contribution)**.** *Evidence coverage is an assurance-relevant structural dimension.*

Construction. Let the claim-relevant action surface be

$$A_C = \{a_1, a_2, \ldots, a_n\}. \tag{73}$$

Let S_E^- and S_E^+ provide equivalent evidence for every event observed by both systems.

System S_E^- exposes evidence only for

$$A^- \subset A_C, \tag{74}$$

while S_E^+ exposes corresponding evidence across all of A_C.

Choose

$$a_j \in A_C \setminus A^-, \tag{75}$$

and let C_E concern whether an unauthorized action occurred anywhere in A_C.

Even if observations over A^- are equally observable, traceable, independently corroborated, and integrity-protected in both systems, S_E^- provides no corresponding evidence capability over a_j.

System S_E^+ therefore enables at least one claim-material capability absent from S_E^-:

$$g_E = \text{“investigate occurrence of unauthorized action } a_j\text{”}. \tag{76}$$

Thus,

$$\mathcal{G}_{q_E}(S_E^-) \neq \mathcal{G}_{q_E}(S_E^+). \tag{77}$$

Evidence coverage is therefore assurance-relevant independently of the quality of

evidence over the already-covered surface. □

If coverage is expanded without losing existing capabilities,

$$S_E^+ \succ_{q_E} S_E^-. \tag{78}$$

4.16 What the Separating Constructions Establish

Propositions 2–8 present candidate separating arguments for the seven structural dimensions.

A completed separating construction has two obligations: a difference in the designated structural profile, and a claim-material difference in the evidentiary capabilities available under the specified assurance context.

The second obligation is not a consequence of the first. It must be established separately.

The companion Lean 4 formalization, version 0.4.1, provides dimension-specific, profile-grounded constructions for observability, evidence coverage, traceability, and reconstructability. These constructions require capability non-equivalence as a separate hypothesis.

Controllability, independence, and integrity remain supported by the informal arguments presented here.

For any candidate dimension d, a completed separation requires

$$\mathcal{G}_{q_d}(S_d^-) \neq \mathcal{G}_{q_d}(S_d^+). \tag{79}$$

Such a result establishes contribution in the specified context. It does not establish universal monotonicity:

$$\forall q,\ d(S_1) > d(S_2) \Rightarrow S_1 \succ_q S_2. \tag{80}$$

Indeed, the notation $d(S_1) > d(S_2)$ may itself be inappropriate for dimensions without a natural numerical scale.

Nor does a separating construction establish that the dimension is required for every assurance claim.

The contribution criterion is existential: a dimension matters to comparative assurability when a controlled structural difference is accompanied by an independently justified difference in claim-material evidentiary capabilities.

The formalization therefore provides a basis for testing candidate dimensions without asserting completeness, independence, or universal necessity.

4.17 Contribution Is Not Sufficiency

The preceding propositions should not be combined into a sufficiency claim.

Let

$$D_S^{(0)} = \{O, C, T, R, N, I, E\} \tag{81}$$

denote the current candidate structural set.

For each candidate dimension, establishing contribution requires a separating construction satisfying

$$\exists q_d, S_d^-, S_d^+ : S_d^- \not\sim_{q_d} S_d^+. \tag{82}$$

This obligation includes an independently justified capability difference.

Even when satisfied, these obligations do not establish

$$D_S^{(0)} \Longrightarrow \text{complete characterization of assurability,} \tag{83}$$

and they do not establish

$$\bigwedge_{d \in D_S^{(0)}} d \Longrightarrow \text{sufficient assurance.} \tag{84}$$

The first unsupported implication would assert completeness of the structural model.

The second would conflate structural evidence capacity with the downstream truth or justification of an assurance claim.

Neither assertion is required by the theory.

4.18 Context Dependence

The same structural difference may affect assurability under one comparative assurance context while being irrelevant under another.

For example, additional internal observability may introduce important evidentiary capabilities for a claim concerning internal computational state while providing no additional capability for a claim resolvable entirely through independently observable external effects.

Likewise, independent corroboration may substantially affect assurability when the original evidence source falls within the threat model, while contributing no material distinction under a context that assumes the source to be trusted.

Thus,

$$S_1 \succ_{q_1} S_2 \tag{85}$$

does not imply

$$S_1 \succ_{q_2} S_2. \tag{86}$$

Nor does

$$S_1 \not\succ_{q_1} S_2 \tag{87}$$

imply

$$S_1 \not\succ_{q_2} S_2. \tag{88}$$

Assurability comparison is inherently relative to the assurance claim and context.

4.19 Access Expansion

Access conditions affect obtainable evidence.

Let

$$K_1 \preceq_K K_2 \tag{89}$$

mean that every evidence source accessible under K_1 remains accessible under K_2, while K_2 may provide additional access.

Holding C, B, the system, and applicable inferential standards fixed, suppose the additional access can be exercised within B.

Then

$$\mathcal{E}_{(C,K_1,B)}(S) \subseteq \mathcal{E}_{(C,K_2,B)}(S). \tag{90}$$

Access expansion therefore cannot reduce the set of obtainable evidence under these assumptions.

It does not follow that

$$\mathcal{G}_{(C,K_1,B)}(S) \subset \mathcal{G}_{(C,K_2,B)}(S). \tag{91}$$

Additional evidence may enable no additional claim-material capability. It may be irrelevant, redundant, invalid for the intended inference, dependent upon an existing source, or otherwise incapable of distinguishing alternatives material to C.

Therefore,

$$\boxed{\text{more access} \Rightarrow \text{no less obtainable evidence}} \tag{92}$$

under the stated assumptions, but

$$\boxed{\text{more obtainable evidence} \not\Rightarrow \text{strictly greater assurability.}} \tag{93}$$

This distinction prevents telemetry volume from being confused with structural assurance capability.

4.20 Resource Dependence

Structural capabilities also interact with resource constraints.

Let

$$\text{cost}(e, S, K) \tag{94}$$

represent the resources required to obtain evidence e from or about S under access conditions K.

The quantity need not be monetary. It may represent computation, time, evaluator effort, storage, latency, specialized expertise, disclosure burden, operational disruption, or another resource constrained by B.

Evidence e belongs to the practically obtainable set only when its acquisition satisfies the applicable resource constraint:

$$\text{cost}(e, S, K) \preceq B. \tag{95}$$

Consequently, two systems may make the same type of evidence theoretically accessible while differing in practical assurability.

Suppose evidence capability g requires evidence e, and

$$\text{cost}(e, S_1, K) \preceq B \tag{96}$$

while

$$\text{cost}(e, S_2, K) \not\preceq B. \tag{97}$$

Then g may belong to

$$\mathcal{G}_q(S_1) \tag{98}$$

while being absent from

$$\mathcal{G}_q(S_2). \tag{99}$$

Thus, architectural differences that make evidence inexpensive, routine, and operationally useful can affect assurability even when comparable evidence is theoretically recoverable from another system given unconstrained resources.

Resource constraints are therefore part of the assurance context rather than an incidental implementation concern.

4.21 Dominance and Incomparability

The capability-set formulation makes dominance explicit.

If

$$\mathcal{G}_q(S_2) \subset \mathcal{G}_q(S_1), \tag{100}$$

then

$$S_1 \succ_q S_2. \tag{101}$$

System S_1 is a strict assurability extension of S_2 for that context: it preserves every claim-material evidentiary capability available from S_2 and adds at least one more.

However, suppose

$$g_a \in \mathcal{G}_q(S_1) \setminus \mathcal{G}_q(S_2) \tag{102}$$

and

$$g_b \in \mathcal{G}_q(S_2) \setminus \mathcal{G}_q(S_1). \tag{103}$$

Then neither capability set contains the other, and

$$S_1 \parallel_q S_2. \tag{104}$$

For example, one architecture may enable detailed internal-state examination while another provides stronger independently generated evidence of externally consequential actions.

If both capabilities are material to C, the general framework need not decide that one architecture is globally superior.

The systems are incomparable under q.

The result indicates an assurance tradeoff.

4.22 Why No Universal Scalar Score Is Required

The framework therefore does not define a quantity such as

$$\mathcal{A}(S) = 0.87. \tag{105}$$

Such a scalar would require additional commitments that have not been established:

1. a complete set of assurance-relevant structural dimensions;
2. valid measurement scales for those dimensions;
3. a mapping from dimensions to evidentiary capabilities;
4. a weighting or aggregation rule for heterogeneous capabilities; and
5. a justification for the tradeoffs implied by that aggregation.

None is required for comparative assurability.

Set containment already permits rigorous statements of equivalence and dominance:

$$\mathcal{G}_q(S_1) = \mathcal{G}_q(S_2), \tag{106}$$

$$\mathcal{G}_q(S_1) \subset \mathcal{G}_q(S_2), \tag{107}$$

or incomparability when neither inclusion holds.

A scalar aggregation could later be introduced for a specific application, assurance authority, procurement regime, or engineering objective.

Such an aggregation would be an additional decision model, not part of the foundational definition of assurability.

4.23 Derived Capabilities and Primitive Dimensions

The distinction between structural dimensions and evidentiary capabilities also clarifies why properties such as reproducibility, auditability, monitorability, testability, and diagnosability need not initially be treated as primitive dimensions.

A derived capability may depend upon several structural dimensions.

For example, a capability

$$g_{\text{replay}} = \text{“reproduce a relevant execution”} \tag{108}$$

may require some combination of:

$$\text{traceability}, \quad \text{reconstructability}, \quad \text{controllability}, \quad \text{integrity}, \tag{109}$$

together with preservation of configuration and environmental state.

The formal model therefore need not assign one primitive dimension to every familiar assurance concept.

Structural dimensions characterize properties of the evidence-producing architecture.

Evidentiary capabilities characterize what that structure makes possible for a

particular assurance problem.

The mapping

$$D_S \longrightarrow \mathcal{E}_q(S) \longrightarrow \mathcal{G}_q(S) \tag{110}$$

may be many-to-many.

This distinction also supplies a test for whether a proposed new concept should be treated as a primitive structural dimension or as a capability derived from existing dimensions.

4.24 Extension of the Structural Model

Let the current structural candidate set be

$$D_S^{(0)} = \{O, C, T, R, N, I, E\}. \tag{111}$$

Suppose future work identifies structural property d^* not represented by $D_S^{(0)}$.

The property should be admitted as an additional assurance-relevant dimension if there exists a separating construction

$$S_{d^*}^-, \qquad S_{d^*}^+, \qquad q_{d^*} \tag{112}$$

such that

$$S_{d^*}^- \not\sim_{q_{d^*}} S_{d^*}^+, \tag{113}$$

and the resulting capability difference cannot already be represented by variation in the existing dimensions.

The structural set can then be extended:

$$D_S^{(1)} = D_S^{(0)} \cup \{d^*\}. \tag{114}$$

Conversely, if a proposed dimension produces no separating construction independent of the existing dimensions, its treatment as a distinct primitive dimension should be reconsidered.

This yields two complementary tests:

> **Admission test.** Does the proposed dimension expose an assurance-relevant capability distinction that the current structural model cannot represent?
>
> **Reduction test.** Can the apparent contribution of the proposed dimension be represented completely through existing structural dimensions?

The first supports extension.

The second supports simplification.

The theory therefore remains explicitly revisable as assurance science and AI architectures evolve.

4.25 Structural Neutrality

The capability ordering does not encode whether an assurance claim receives a favorable outcome.

Suppose

$$S_1 \succ_q S_2. \tag{115}$$

This means that S_1 enables a strict superset of the claim-material evidentiary capabilities available from S_2.

It does not imply

$$C(S_1) \tag{116}$$

is more likely to be true than

$$C(S_2). \tag{117}$$

Indeed, the additional capability may expose evidence that defeats C.

Thus,

$$\boxed{S_1 \succ_q S_2 \not\Rightarrow \text{a more favorable assurance conclusion for } S_1.} \tag{118}$$

This is an essential neutrality property of the framework.

Structural assurability describes the capacity for legitimate examination. It does not encode the desired result of that examination.

4.26 Summary of Formal Claims

The formal contribution of this section is intentionally bounded.

We establish the following.

1. For a fixed comparative assurance context $q = (C, K, B)$, a system has a set of practically obtainable evidence

   $$\mathcal{E}_q(S),$$

   a claim-material subset

   $$\mathcal{E}_q^C(S),$$

   and a corresponding set of evidentiary capabilities

   $$\mathcal{G}_q(S).$$

2. Evidence sets are compared through the claim-relative capability map

   $$\Gamma_C(E),$$

   rather than through raw evidence volume or artifact identity.

3. Comparative assurability is defined by capability containment:

$$S_1 \succeq_q S_2 \quad \Longleftrightarrow \quad \mathcal{G}_q(S_2) \subseteq \mathcal{G}_q(S_1).$$

4. Behavioral equivalence does not imply assurability equivalence.
5. A candidate structural dimension contributes to assurability when it admits a separating construction for which

$$S_d^- \not\sim_{q_d} S_d^+.$$

6. Strict dominance,

$$S_d^+ \succ_{q_d} S_d^-,$$

is a stronger construction-specific result requiring preservation of the weaker system's claim-material capabilities together with at least one additional capability.
7. Seven candidate dimensions are examined through illustrative separating arguments. Four have corresponding dimension-specific, profile-grounded constructions in the companion Lean 4 formalization.
8. Establishing contribution requires both structural-profile separation and capability non-equivalence. Neither obligation establishes universal necessity, completeness, or sufficiency.
9. The assurability relation permits equivalence, strict dominance, and incomparability without requiring a universal scalar score.
10. Additional structural dimensions may be admitted when they expose assurance-relevant capability distinctions not representable by the existing dimensions.

The resulting theory separates three questions that are frequently collapsed:

$$\boxed{\begin{array}{c}\text{What evidence does the structure make obtainable?}\\ \downarrow \\ \text{What assurance-relevant capabilities does that evidence enable?}\\ \downarrow \\ \text{What conclusion should an assurance argument draw?}\end{array}} \tag{119}$$

Structural assurability addresses the first two while deliberately leaving the final

inferential judgment to the assurance process.

The following section applies the framework to representative AI architectures to determine whether systems with similar functional capabilities can occupy different positions in the comparative assurability relation.

5 Applying Structural Assurability to AI Architectures

The preceding sections developed structural assurability as a claim-relative property concerning the capacity of a system and its evidence-producing environment to make assurance-relevant distinctions possible. This section applies that framework to representative AI architectures.

The purpose is not to evaluate particular commercial systems or prescribe a preferred architecture. Instead, the examples examine whether the formal distinctions developed in Sections 3 and 4 reveal meaningful differences among systems that may otherwise appear functionally similar.

Three cases are considered.

The first demonstrates strict comparative dominance for a claim concerning unauthorized consequential action.

The second demonstrates assurability equivalence for a claim that can be resolved primarily through externally observable behavior.

The third demonstrates incomparability when different architectures enable different assurance-relevant distinctions.

Together, the cases illustrate that structural assurability is claim-relative and need not induce a universal ranking of architectures.

5.1 Representative Architectures

Consider three abstract AI system architectures.

They are intentionally simplified. Each represents a class of evidence-producing

structure rather than a specific implementation.

5.1.1 Opaque Service Architecture

Let S_O denote an AI system exposed primarily through an input-output interface.

The system accepts requests and returns outputs. Standard application-level records may establish that a request occurred and that a response was returned, but the assurance interface exposes little or no trusted evidence concerning internal execution, tool use, authorization decisions, retrieved information, intermediate state, or causal provenance.

A simplified evidence surface is

$$S_O : \quad \{\text{input}, \text{output}, \text{request metadata}\}. \tag{120}$$

This architecture is not assumed to be unsafe or unreliable. It is simply structurally limited in the assurance-relevant distinctions that can be made from its exposed evidence.

5.1.2 Instrumented Agent Architecture

Let S_A denote an agentic AI system with runtime instrumentation.

In addition to externally visible inputs and outputs, the system records evidence concerning consequential execution events, such as

$$\begin{aligned} S_A : \quad & \{\text{input}, \text{output}, \text{tool invocations}, \text{retrieval events}, \\ & \text{authorization decisions}, \text{memory updates}, \text{state transitions}, \text{provenance}\}. \end{aligned} \tag{121}$$

The architecture therefore provides a larger evidence surface concerning system execution.

However, the evidence may still be generated and preserved within infrastructure controlled by the same operator or execution environment as the agent itself.

5.1.3 Assurability-Oriented Architecture

Let S_R denote an architecture explicitly designed to preserve assurance-relevant evidence.

It includes the runtime evidence available from S_A and adds mechanisms such as

$$\begin{aligned} S_R : \quad & \{\text{instrumented execution evidence}, \text{independent action observation}, \\ & \text{tamper-evident evidence preservation}, \text{authenticated provenance}, \\ & \text{controlled replay or intervention}, \text{explicit action-surface coverage}, \\ & \text{selected internal-state instrumentation}\}. \end{aligned} \tag{122}$$

The term *assurability-oriented* is descriptive rather than normative. The architecture is designed to increase the range of assurance-relevant distinctions that can be made under specified conditions.

It does not follow that the system satisfies any particular assurance claim.

5.2 Case 1: Unauthorized Consequential Action

Consider the assurance claim

$$\begin{aligned} C_1 = \text{“The agent performed no unauthorized consequential action} \\ \text{during interval } T\text{.”} \end{aligned} \tag{123}$$

Suppose a consequential external action a is observed during T.

Relevant assurance questions include:

$Q_1 =$ Did action a occur?

$Q_2 =$ Which component or actor initiated a?

$Q_3 =$ What authorization state applied when a was initiated?

$Q_4 =$ Through what execution pathway did a occur?

$Q_5 =$ Could the evidence concerning a have been altered or suppressed?

$Q_6 =$ Is the evidence concerning a corroborated by a source
outside the evaluated component?

$Q_7 =$ Were all consequential-action channels represented by the evidence surface?

These questions define claim-relevant distinctions.

For example, Q_2 may require distinguishing

$$\text{agent initiated } a \quad \text{from} \quad \text{human initiated } a, \tag{124}$$

while Q_3 may require distinguishing

$$\text{authorized action} \quad \text{from} \quad \text{unauthorized action.} \tag{125}$$

5.2.1 Opaque Service

Under S_O, external evidence may establish that a result occurred, but the evidence surface may not permit reliable attribution of the action to the agent, a human operator, an external service, or another component.

Thus, the architecture may permit resolution of Q_1 while leaving Q_2–Q_7 substantially unresolved.

Its claim-relevant distinction set may therefore be represented schematically as

$$\mathcal{G}_{q_1}(S_O) = \{g_{\text{occurrence}}\}. \tag{126}$$

5.2.2 Instrumented Agent

System S_A additionally exposes runtime traces linking actions to tool invocations, authorization decisions, retrieval events, and state transitions.

It may therefore permit distinctions concerning:

$$\begin{aligned} &\text{action occurrence},\\ &\text{action initiator},\\ &\text{authorization state},\\ &\text{execution pathway}. \end{aligned} \tag{127}$$

Schematically,

$$\mathcal{G}_{q_1}(S_A) = \{g_{\text{occurrence}}, g_{\text{origin}}, g_{\text{authorization}}, g_{\text{pathway}}\}. \tag{128}$$

However, if all records are generated and preserved within the same administrative or computational boundary as the agent, questions concerning evidence manipulation or independent corroboration may remain unresolved.

5.2.3 Assurability-Oriented Architecture

System S_R preserves the distinctions available from S_A while adding independently generated observations, tamper-evident records, authenticated provenance, and explicit monitoring of consequential-action channels.

It may therefore additionally distinguish:

$$\text{unaltered evidence} \quad \text{from} \quad \text{evidence subject to undetected manipulation}, \tag{129}$$

and

$$\text{complete monitored action surface} \quad \text{from} \quad \text{partially observed action surface}. \tag{130}$$

Schematically,

$$\begin{aligned}\mathcal{G}_{q_1}(S_R) = \{&g_{\text{occurrence}}, g_{\text{origin}}, g_{\text{authorization}}, g_{\text{pathway}}, \\ &g_{\text{integrity}}, g_{\text{coverage}}, g_{\text{independent corroboration}}\}.\end{aligned} \tag{131}$$

If each architecture preserves the capabilities of the preceding architecture while adding the capabilities shown above, then

$$\mathcal{G}_{q_1}(S_O) \subset \mathcal{G}_{q_1}(S_A) \subset \mathcal{G}_{q_1}(S_R). \tag{132}$$

Therefore,

$$S_R \succ_{q_1} S_A \succ_{q_1} S_O. \tag{133}$$

This ordering is not a general ranking of the three architectures.

It holds only for the specified claim and assurance context.

The result illustrates how structural differences can create strict comparative assurability without establishing that any of the systems actually satisfies C_1.

Indeed, S_R may provide stronger evidence that an unauthorized action *did* occur.

5.3 Case 2: Externally Observable Output Property

Consider a different claim:

$$\begin{aligned}C_2 = \text{“The response returned for request } x \text{ contains no} \\ \text{instance of prohibited output class } P\text{.”}\end{aligned} \tag{134}$$

Assume that membership in P can be determined completely and reliably from the

externally returned response.

The material assurance question is then simply whether

$$y \in P, \tag{135}$$

where y is the externally observable output.

All three architectures expose y.

Additional information concerning tool calls, internal execution, provenance, independent runtime observation, or state transitions may be valuable for other purposes, but under the assumptions of this claim it enables no additional distinction required to determine whether $y \in P$.

Thus,

$$\mathcal{G}_{q_2}(S_O) = \mathcal{G}_{q_2}(S_A) = \mathcal{G}_{q_2}(S_R). \tag{136}$$

Therefore,

$$S_O \sim_{q_2} S_A \sim_{q_2} S_R. \tag{137}$$

This case is important because it prevents a common misinterpretation of structural assurability.

More instrumentation does not necessarily imply greater assurability.

If the assurance-relevant distinction can already be resolved completely from externally available evidence, additional structural evidence may contribute nothing to that particular assurance context.

Thus,

$$S_R \succ_{q_1} S_O \tag{138}$$

and

$$S_R \sim_{q_2} S_O \tag{139}$$

can both hold without contradiction.

Assurability is claim-relative.

5.4 Case 3: Internal Visibility Versus Independent Evidence

The previous examples produced strict dominance and equivalence.

A third case illustrates incomparability.

Consider two architectures, S_V and S_N.

System S_V provides extensive visibility into internal execution. It exposes selected intermediate representations, internal state transitions, detailed tool-selection processes, and model-generated explanations of relevant actions.

However, these observations are produced and preserved by infrastructure controlled by the same operator and execution environment as the system under evaluation.

System S_N, by contrast, exposes little internal state. It provides only limited visibility into model computation.

However, consequential external actions are independently observed by a separate enforcement or monitoring component. Those records are authenticated and protected from retrospective alteration by the evaluated system.

Consider the claim

$$C_3 = \text{“The agent intentionally attempted an unauthorized consequential action.”} \tag{140}$$

The claim contains at least two materially different components.

One concerns internal process:

$$q_{\text{internal}} = \begin{array}{l}\text{Did the agent enter a state or process} \\ \text{relevant to attempted unauthorized action?}\end{array} \tag{141}$$

The other concerns external execution and attribution:

$$q_{\text{external}} = \begin{array}{l}\text{Did a consequential unauthorized action occur,} \\ \text{and can it be independently attributed?}\end{array} \tag{142}$$

System S_V may enable distinctions over internal computational behavior unavailable from S_N:

$$g_{\text{internal}} \in \mathcal{G}_{q_3}(S_V) \setminus \mathcal{G}_{q_3}(S_N). \tag{143}$$

System S_N, however, may enable independently corroborated distinctions concerning consequential external action that S_V cannot provide:

$$g_{\text{independent}} \in \mathcal{G}_{q_3}(S_N) \setminus \mathcal{G}_{q_3}(S_V). \tag{144}$$

Accordingly,

$$\mathcal{G}_{q_3}(S_V) \not\subseteq \mathcal{G}_{q_3}(S_N) \tag{145}$$

and

$$\mathcal{G}_{q_3}(S_N) \not\subseteq \mathcal{G}_{q_3}(S_V). \tag{146}$$

Therefore,

$$S_V \parallel_{q_3} S_N. \tag{147}$$

Neither system dominates.

A scalar assurance score would require assigning a relative value to internal visibility

and independent external evidence.

The structural framework does not impose such a tradeoff.

That decision belongs to the specific assurance process, threat model, or decision authority.

5.5 Recasting the Cases as Claim-Relevant Distinctions

The three cases can be interpreted through a common principle.

Let Ω_C represent alternatives relevant to assurance claim C.

Evidence contributes to structural assurability when it permits distinctions among alternatives in Ω_C that would otherwise remain unresolved.

For Case 1, relevant alternatives include:

$$\begin{aligned}\Omega_{C_1} = \{&\text{no consequential action},\\ &\text{authorized agent action},\\ &\text{unauthorized agent action},\\ &\text{human-initiated action},\\ &\text{externally initiated action},\\ &\text{action with unreliable historical record}\}.\end{aligned} \tag{148}$$

The richer architectures expose evidence permitting progressively more of these alternatives to be distinguished.

For Case 2,

$$\Omega_{C_2} = \{y \in P, y \notin P\}. \tag{149}$$

The externally visible response already permits the relevant distinction. Additional structural instrumentation need not increase assurability.

For Case 3, the relevant alternatives span both internal-process hypotheses and externally observable action hypotheses. Different architectures distinguish different

subsets, producing incomparability.

These examples motivate a more general interpretation:

> Structural assurability concerns the capacity of a system and its evidence-producing environment to make distinctions among alternatives material to an assurance claim.

Under this interpretation, the structural dimensions introduced in Section 3.5 are important because removing one can make an assurance-relevant distinction unavailable.

5.6 Relationship to the Structural Dimensions

The examples also illustrate how the candidate structural dimensions affect assurance-relevant distinctions.

Observability. Observability determines whether relevant states, events, or effects enter the available evidence surface at all.

Controllability. Controllability determines whether an evaluator can create or reproduce conditions required to distinguish claim-relevant alternatives.

Traceability. Traceability permits observed events to be distinguished according to origin, causal pathway, or transformation history.

Reconstructability. Reconstructability preserves distinctions that may be required after the original execution has ended.

Independence. Independence permits distinctions between system self-report and corroboration generated outside the component or actor being evaluated.

Integrity. Integrity permits distinctions between trustworthy preservation of historical evidence and histories subject to undetected alteration or suppression.

Evidence Coverage. Evidence coverage determines which portions of the claim-relevant state, event, or action surface admit evidentiary distinctions at all.

The dimensions therefore need not be interpreted as components of a numerical assurability score.

Their more fundamental role is to govern which claim-relevant distinctions the evidence-producing architecture makes possible.

5.7 Cost and Practical Assurability

The examples above can also be extended to include resource constraints.

Suppose S_A and S_R both permit reconstruction of a consequential execution.

In S_A, reconstruction requires several days of specialist analysis across heterogeneous logs.

In S_R, authenticated provenance and preserved execution state permit the same claim-relevant distinction to be established automatically within minutes.

Under an unconstrained budget, both systems may enable the same evidentiary capability.

Under a resource constraint B^*,

$$\operatorname{cost}\left(g_{\text{reconstruction}}, S_A, K\right) > B^* \tag{150}$$

while

$$\operatorname{cost}\left(g_{\text{reconstruction}}, S_R, K\right) \leq B^*. \tag{151}$$

Then

$$g_{\text{reconstruction}} \notin \mathcal{G}_{(C,K,B^*)}(S_A) \tag{152}$$

while

$$g_{\text{reconstruction}} \in \mathcal{G}_{(C,K,B^*)}(S_R). \tag{153}$$

The systems therefore differ in practical assurability despite admitting similar evidence in principle.

This distinction is particularly important for deployed AI systems. Assurance mechanisms that are technically possible but prohibitively expensive, slow, fragile, or operationally disruptive may provide little practical assurance under realistic constraints.

Structural features that make useful evidence inexpensive and routine can therefore expand practical assurability without changing the underlying assurance claim.

5.8 What the Cases Demonstrate

The three cases establish several properties of the framework.

First, systems exhibiting similar functional behavior can differ materially in the assurance-relevant distinctions supported by their evidence-producing structures.

Second, additional instrumentation does not automatically increase assurability. Its value depends upon the assurance claim.

Third, structural assurability is not a universal ranking of architectures. The same pair of systems may exhibit strict dominance under one claim and equivalence under another.

Fourth, architectures may be incomparable when each exposes claim-material distinctions unavailable from the other.

Fifth, independence, integrity, provenance, internal visibility, and coverage are not interchangeable forms of "more evidence." They can enable qualitatively different assurance-relevant distinctions.

Sixth, resource constraints can convert differences in the efficiency of evidence acquisition into differences in practical assurability.

Most importantly, the cases support the central interpretation developed in this work:

> A system is structurally more assurable in a specified assurance context when its architecture and evidence-producing environment permit assurance-relevant distinctions that cannot practically be made for the comparison system, while preserving the distinctions already available from that system.

This statement remains deliberately neutral regarding the eventual assurance outcome.

A richer evidence structure may make it easier to support an assurance claim, easier to refute it, or both.

Structural assurability concerns the capacity for examination, not the conclusion of that examination.

The examples therefore reinforce the separation

$$\boxed{\begin{aligned}&\text{System Structure} \to \text{Obtainable Evidence}\\ &\qquad\qquad\qquad\to \text{Assurance-Relevant Distinctions} \to \text{Assurance Reasoning}.\end{aligned}} \tag{154}$$

The present framework addresses the first three terms.

The final inference from evidence and distinctions to an assurance conclusion remains the responsibility of the applicable assurance methodology.

6 Theoretical Results

The definitions and results of Sections 3 and 4 are developed in a Lean 4 formalization. The companion Lean 4 formalization at version 0.4.1 is authoritative for formal statements and proofs (Case 2026b).

6.1 Established in the Formalization

- *Resolution bound.* If two admissible worlds are observationally indistinguishable under an observation model and disagree on a claim, the model does not

resolve the claim.

- *Capability comparison.* Capability sets are compared by containment. The capability model is abstract. Monotonicity of capabilities under added evidence is an explicit assumption, not a derived result.
- *Access and resource expansion.* The formalization states results on expanding evaluator access and resource bounds, and a conservative-extension result.
- *Separating constructions.* A generic separating construction is stated, together with dimension-specific constructions for observability, evidence coverage, traceability, and reconstructability. Each construction involves two obligations: the two systems differ in the named dimension's profile, and their capability sets are not equivalent. The second is a separate hypothesis, not a consequence of the first.

6.2 Not Established

- Independence, integrity, and controllability are defined as candidate dimensions but have no dimension-specific separating construction.
- The separating-construction criterion establishes contribution to assurance-relevant capability, not necessity or sufficiency.
- The capability model is not derived from claim resolution beyond the worked examples.
- The finite-model experiments on observability gaps, integrity, and collusion are exploratory and are not part of the formal theory.

7 Illustrative Application

A companion pilot repository applies the resolution bound to an external evaluation framework. Its initial study uses the query-violation example in NIST's initial public draft of the TEVV-Athlon framework, NIST AI 200-2 (Phillips et al. 2026). The pilot adds two researcher-defined claims to that example. NIST does not assert either claim.

In a finite model where organic behavior in four scenarios varies independently, observations from the three tested scenarios do not resolve a claim that also covers the fourth, with or without Red Teaming observations. If organic and adversarial

behavior vary independently, Red Teaming alone does not resolve an organic-use claim on the three tested scenarios. Each result comes with a pair of indistinguishable worlds that disagree on the claim.

Because the model admits every combination of scenario outcomes, these results follow from its admissibility assumptions. They illustrate the resolution bound; they do not test the theory. They make no claim about operational AI systems. Comparison of evaluator contexts by capability-set containment has not yet been carried out. The pilot record remains open.

8 Discussion

Structural assurability is claim-relative. The same system can support one claim well and another poorly. Comparison is by containment of capability sets, so incomparability is a possible and informative outcome.

The framework complements assurance cases, auditability, monitorability, and evaluation-rigor methods. It addresses what evidence an architecture and its environment make obtainable. It does not address how that evidence is argued or how much confidence it warrants.

Three extensions remain open: formal grounding of independence, integrity, and controllability; grounding of the capability model in claim resolution; and comparison of evaluator contexts for an external framework.

9 Threats to Validity and Limitations

Construct validity. The seven dimensions are provisional. The separating-construction criterion requires a controlled structural-profile difference and an independently established capability difference. A completed construction demonstrates only the claim-relative contribution, not universal necessity or sufficiency.

Formal model. The capability model is abstract, and monotonicity is assumed. Capability non-equivalence is a separate hypothesis from the profile difference. Only four of the seven dimensions have dimension-specific constructions. The Lean

formalization checks the stated theorems. It does not check that the models are faithful to a real system or claim.

External validity. The worked examples are finite and use table-lookup capabilities. The pilot results are conditional on researcher-defined claims and admissibility assumptions. No operational AI system has been assessed.

Exploratory experiments. The finite-model experiments are illustrations, not evidence about deployed systems.

10 Conclusion

This paper proposed structural assurability: the claim-relative capacity of a system and its evidence-producing environment to make assurance-relevant evidence practically obtainable. Assurability is compared by containment of capability sets, so incomparability is permitted. The accompanying Lean formalization establishes the resolution bound and separating constructions for four of the seven provisional dimensions. Independence, integrity, controllability, and grounding of the capability model in claim resolution remain open.

Statements and Declarations

Acknowledgments

This research made use of SciX, a scientific literature search and discovery platform, whose related-literature tools helped identify relevant work across disciplinary boundaries.

Author Contributions

The author is the sole contributor to this work and is responsible for all aspects of the research, authorship, and publication.

Code Availability

A reference implementation that checks the finite constructions of this paper is available (Case 2026a).

Use of AI-Assisted Tools

AI-assisted tools were used for editing, formatting, and consistency checking, and in developing and reviewing mathematical definitions, proofs, and the companion Lean 4 formalization. The author reviewed all suggestions and is solely responsible for the content.

Declaration of Conflicting Interest

The author confirms there are no potential conflicts of interest.

References

Miron Abramovici, Melvin A. Breuer, and Arthur D. Friedman. *Digital Systems Testing and Testable Design*. Wiley-IEEE Press, 1990. URL `https://ieeexplore.ieee.org/servlet/opac?bknumber=5266057`.

Raghavendra Sai Akkinapragada. Three classes of AI accountability systems: A taxonomy of opaque, explainable, and replayable architectures. SSRN, June 2026. URL `https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6943184`. Preprint.

Ross Anderson. *Security Engineering: A Guide to Building Dependable Distributed Systems*. Wiley, Indianapolis, IN, 3 edition, 2020. ISBN 978-1-119-64278-7.

Apollo Research. The need for deeper, white-box access to maintain state of the art evaluations for loss of control threats. Apollo Research governance publication, May 2026. URL `https://www.apolloresearch.ai/governance/the-need-for-deeper-white-box-access-to-maintain-state-of-the-art-evaluations-for-loss-of-`. Accessed 2026-11-08.

Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 4 edition, 2021. ISBN 978-0-13-688609-9.

Robert V. Binder. Design for testability in object-oriented systems. *Communications of the ACM*, 37(9):87–101, September 1994. doi: 10.1145/182987.184077.

Robin E. Bloomfield and Peter G. Bishop. Safety and assurance cases: Past, present and possible future—an adelard perspective. In Chris Dale and Tom Anderson, editors, *Making Systems Safer: Proceedings of the Eighteenth Safety-Critical Systems Symposium*, pages 51–67. Springer, 2010. doi: 10.1007/978-1-84996-086-1_4.

Markus Borg, Jens Henriksson, Kasper Socha, Olof Lennartsson, Elias Sonnsjö Lönegren, Thanh Bui, Piotr Tomaszewski, Sankar Raman Sathyamoorthy, Sebastian Brink, and Mahshid Helali Moghadam. Ergo, SMIRK is safe: A safety case for a machine learning component in a pedestrian automatic emergency brake system. *Software Quality Journal*, 31:335–403, 2023. doi: 10.1007/s11219-022-09613-1.

Trenton Bricken, Adly Templeton, et al. Towards monosemanticity: Decomposing language models with dictionary learning. Transformer Circuits Thread, October 2023. URL `https://transformer-circuits.pub/2023/monosemantic-features/index.html`.

Simon Burton. A causal model of safety assurance for machine learning, 2022. URL `https://arxiv.org/abs/2201.05451`.

Denise M. Case. Operational identity: A finite audit of declared and implemented rules of sameness, 2026a. URL `https://arxiv.org/abs/2607.20729`.

Denise M. Case. Formal theory: Structural assurability. Zenodo; Lean 4 formalization, October 2026b. URL `https://github.com/structural-explainability/se-theory-structural-assurability/tree/v0.4.1`. Version 0.4.1, released 2026-10-30; Apache-2.0.

Paul C. Clements, Rick Kazman, and Mark H. Klein. *Evaluating Software Architectures: Methods and Case Studies*. Addison-Wesley Professional, 2002. ISBN 978-0-201-70482-2.

Common Criteria Recognition Arrangement. Common criteria for information technology security evaluation. Technical report, ISO, 2022a. CC:2022 Release 1.

Common Criteria Recognition Arrangement. Common criteria for information technology security evaluation: Part 5: Pre-defined packages of security requirements. Technical Report CCMB-2022-11-005, Common Criteria Recognition Arrangement, November 2022b. URL `https://www.commoncriteriaportal.org/files/ccfiles/CC2022PART5R1.pdf`. CC:2022 Revision 1.

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. Transformer Circuits Thread, September 2022. URL `https://transformer-circuits.pub/2022/toy_model/index.html`.

Linda Fernsel, Yannick Kalff, and Katharina Simbeck. Assessing the auditability of AI-integrating systems: A framework and learning analytics case study, 2024. Preprint.

Roy S. Freedman. Testability of software components. *IEEE Transactions on Software Engineering*, 17(6):553–564, June 1991. doi: 10.1109/32.87281.

Arthur Goemans, Marie Davidsen Buhl, Jonas Schuett, Tomek Korbak, Jessica Wang, Benjamin Hilton, and Geoffrey Irving. Safety case template for frontier AI: A cyber inability argument, 2024.

John B. Goodenough, Charles B. Weinstock, and Ari Z. Klein. Toward a theory of assurance case confidence. Technical Report CMU/SEI-2012-TR-002, Software Engineering Institute, Carnegie Mellon University, September 2012.

Patrick J. Graydon and C. Michael Holloway. An investigation of proposed techniques for quantifying confidence in assurance arguments. *Safety Science*, 92:53–65, 2017. doi: 10.1016/j.ssci.2016.09.014.

Melody Y. Guan, Miles Wang, Micah Carroll, Zehao Dou, Annie Y. Wei, Marcus Williams, Benjamin Arnav, Joost Huizinga, Ian Kivlichan, Mia Glaese, Jakub Pachocki, and Bowen Baker. Monitoring monitorability, December 2025. OpenAI publication page: "Evaluating chain-of-thought monitorability", 2025-12-18; ICML 2026.

Richard Hawkins, Colin Paterson, Chiara Picardi, Yan Jia, Radu Calinescu, and Ibrahim Habli. Guidance on the assurance of machine learning in autonomous systems (AMLAS). *arXiv preprint arXiv:2102.01564*, 2021.

IEEE. Design for testability. IEEE Technical Topic, 2026. https://technav.ieee.org/topic/design-for-testability/.

Joint Committee for Guides in Metrology. Evaluation of measurement data—guide to the expression of uncertainty in measurement. Technical Report JCGM 100:2008, Joint Committee for Guides in Metrology, 2008.

Rick Kazman, Mark H. Klein, and Paul C. Clements. ATAM: Method for architecture evaluation. Technical Report CMU/SEI-2000-TR-004, Software Engineering Institute, Carnegie Mellon University, August 2000.

Nancy G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, Cambridge, MA, 2011. ISBN 978-0-262-01662-9.

Grace Lewis, Rachel Brower-Sinning, Robert Edman, Alex Derr, Ipek Ozkaya, and Sebastián Echeverría. A quality model for machine learning components. Software Engineering Institute Blog, July 2026.

Yueqi Li and Sanjay Goel. Making it possible for the auditing of AI: A systematic review of AI audits and AI auditability. *Information Systems Frontiers*, 27(3): 1121–1151, June 2025. doi: 10.1007/s10796-024-10508-8.

Alexander Meinke, Bronson Schoen, Jérémy Scheurer, Mikita Balesni, Rusheb Shah, and Marius Hobbhahn. Frontier models are capable of in-context scheming, 2024.

Samuel Messick. Validity of psychological assessment: Validation of inferences from persons' responses and performances as scientific inquiry into score meaning. *American Psychologist*, 50(9):741–749, 1995. doi: 10.1037/0003-066X.50.9.741.

National Institute of Standards and Technology. Auditability in voting systems. Technical report, National Institute of Standards and Technology, 2003. https://www.nist.gov/system/files/documents/itl/vote/Saltman-AuditabilityofDREs-Revised-2003.pdf.

National Institute of Standards and Technology. Assurance case. NIST Computer Security Resource Center Glossary, 2009. URL `https://csrc.nist.gov/glossary/term/assurance_case`.

National Institute of Standards and Technology. Challenges to the monitoring of deployed ai systems. Technical Report NIST AI 800-4, National Institute of Standards and Technology, March 2026a. Center for AI Standards and Innovation.

National Institute of Standards and Technology. Building evaluation probes into agentic AI. NIST Information Technology Laboratory, AI Program project page, 2026b. URL https://www.nist.gov/programs-projects/building-evaluation-probes-agentic-ai. Accessed 2026-11-08.

Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 5(3):e00024.001, March 2020. doi: 10.23915/distill.00024.001.

OpenTelemetry. AI agent observability—evolving standards and best practices. OpenTelemetry Blog, 2025. URL https://opentelemetry.io/blog/2025/ai-agent-observability/. Accessed 2026-11-08.

OWASP. Agent observability standard (AOS), n.d. URL https://aos.owasp.org/. Accessed 2026-11-08; specification status: working draft.

P. Jonathon Phillips, Theodore Jensen, Patrick Hall, Razvan Amironesei, Yee-Yin Choong, C. Greenberg, and K. K. Green. The TEVV-Athlon framework for evaluating AI systems. Technical Report NIST AI 200-2 ipd, National Institute of Standards and Technology, Gaithersburg, MD, August 2026. URL https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.200-2.ipd.pdf. Initial Public Draft.

RTCA. Software considerations in airborne systems and equipment certification. RTCA DO-178C, December 2011.

John Rushby. Just-in-time certification. In *12th IEEE International Conference on the Engineering of Complex Computer Systems (ICECCS)*, pages 15–24, Auckland, New Zealand, July 2007. IEEE Computer Society.

SAE International. Guidelines for development of civil aircraft and systems. SAE Aerospace Recommended Practice ARP4754A, December 2010.

Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.

Laura Waltersdorfer, Fajar J. Ekaputra, Tomasz Miksa, and Marta Sabou. AuditMAI: Towards an infrastructure for continuous AI auditing, 2024. Preprint.

T. W. Williams and K. P. Parker. Design for testability a survey, January 1982. ISSN 0018-9340. URL https://doi.org/10.1109/TC.1982.1675879.

Roman V. Yampolskiy. On monitorability of AI. *AI and Ethics*, 5(1):689–707, 2025. doi: 10.1007/s43681-024-00420-x.

Xiaofei Zhao. Composable assurance for AI alignment: A framework for propagating formal safety properties through MLOps. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(44):38129–38136, 2026. doi: 10.1609/aaai.v40i44.41151.